National University of Computer and Emerging Sciences

Abstractive Text Summarization using Deep Learning

FYP Team

M. Faiq ……....…18L-2137
Zulqarnain……..…19L-2708
Mustafa Khalid…..…19L-1050

Supervised by
Samin Iftikhar

Co-Supervised by
Aamir Wali

FAST School of Computing

National University of Computer and Emerging Sciences

[Lahore], Pakistan

October 2022

Anti-Plagiarism Declaration

This is to declare that the above publication produced under the:

Title: ____Abstractive Text Summarization using Deep Learning___. _____

is the sole contribution of the author(s) and no part hereof has been reproduced on **as it is** basis (cut and paste) which can be considered as **Plagiarism**. All referenced parts have been used to argue the idea and have been cited properly. I/We will be responsible and liable for any consequence if violation of this declaration is determined.

Date: ___11 October 2022_____

Student 1

Name: _____M. Faiq_____

Signature: ____ _____

Student 2

Name: _____Zulqarnain_____

Signature: _____ _____

Student 3

Name: _____Mustafa Khalid_____

Signature: _____ _____

Authors' Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

The purpose of this project is to develop a news summarization platform using abstractive techniques via deep learning models. We have developed a fully-fledged website in python and Django backend. Furthermore, the models have been developed using TensorFlow and hugging face. In particular, will be focusing on RNNs and Transformers. We developed RNN models using TensorFlow library and transformer models in hugging face. Furthermore, human evaluation is conducted on 2 of our best models from a sample size of 36 individuals. We picked these models since the rogue scores were good and only give a representation of context capturing rather than considering other complexities in language such as grammar and fluency. Lastly, to push the limits of our research, we also created a small technical dataset with 200 tuples. This dataset was designed and tailored to create a multitasking based deep learning model which learns to generate different kinds of outputs for the input tuples guided by control signals in the input. Furthermore, we also tested our baselines on the same dataset and also tested our multitasking on CNN/Daily Mail on which the baselines models were originally trained and tested on. We consider two approaches for this multitasking problem: Topic guided summarization where model learns to generate both topics and abstractive summaries. In addition, also a variation where the model learns to generate abstractive and extractive summaries and compare their relative summaries. Lastly, out of both models, we used abstractive, extractive model to test on our CNN/Daily dataset to see how good its rogue scores are.

Executive Summary

In chapter 1, we introduce the purpose of this document and why it has been created. We introduce the problem that we are trying to solve, give a birds eye view of its importance, its concerned benefactors and appropriate background required to understand the importance of why we have chosen abstractive text summarization technique to solve the overarching problem of text summarization. We also compare it with another contrasting technique known as abstractive text summarization and explain what abstractive text summarization solves that extractive summarization can't. Furthermore, we have also enlisted a handful of applications where summarization is considered a useful utility; especially in cases when summarization needs creativity.

In chapter 2, we discuss the problem domain of abstractive text summarization itself, where it falls in reference to the scope, goals and the objectives of our project. We describe the problem that we are trying to solve, which is generating factually consistent abstractive summaries as if they are written by a human being. Since our project is a research/learning project, we emphasize the exploratory aspect of our project; which is to explore certain deep learning baseline models and compare the quality of their summary generation with one another. We will also explore metrics that can be used for evaluation and where applicable, incorporate human evaluation. As we move further along the project, our goal will be to improve the summaries generated and make them as relevant and factually consistent as possible which is also the main problem statement defined.

The scope of this project will cover following main sections:

1. We will only focus on abstractive text summarization.
2. We will only focus on summarization of text that falls under current affairs.
3. We will only focus on English text summarizations. Other languages are not within the scope of the project.

We also elaborate the research aspects related to the overarching problem of abstractive text summarization and what we will be trying to achieve in this project, what inputs our system will take and the output it generates. Secondly, we also discuss the subproblems we will be solving while implementing the models; which is basically the entire data science pipeline of:

1. Data collection
2. Data pre-processing/Cleaning
3. Vectorisation
4. Model implementation
5. Evaluation

In the literature review section, we address numerous abstractive text summarization techniques using Deep Learning. In particular, we will be exploring seq2seq based models which are variations of standard encoder decoder architecture using attention mechanism. Furthermore, we also take a deep dive into Transformer networks which are considered a novel improvement on the existing encoder decoder architectures using self-attention mechanism. We also take a deep dive into GANs and their variations adapted to the task text to text sequence modelling. Each of these models are explored in reference to existing studies done by other authors extracted from reputable journals and each study is compared and analyzed in reference to some evaluation metric such as Rogue Scores(commonly used throughout the papers).

In chapter 4, we gave a detailed description of the methodology proposed and the models we intend to use for our experiments. We used RNN based Encoder Decoders and Transformers models. For word embeddings we employed Word2Vec and Glove.

Furthermore, in chapter 5, we have provided a detailed discussion of the experiments conducted a detailed set of experiments on both LSTMs and Transformer models and compared their results with one another. We observed that RNNs performed much poorer compared to our two transformer models: T5 and Hybrid. Whereas T5 had the best scoring results out of all 3 major types of models deployed.

In chapter 6, we give a detailed and comprehensive illustration of the High and Low level design followed keeping in view the best practices for the development of our system. We first give an overview of the overarching system, addressing its major components as follows:

1. User Interface
2. Text Summarization Service
3. Database
4. Authentication and Authorization Service

Furthermore, in this chapter, we also describe the low level design of our system by providing a class diagram, keeping in view the major principles of OOP. Lastly, a sequence diagram has also been provided to describe user interaction and application workflow in detail.

In chapter 7, we have also given a detailed overview of the libraries and frameworks used for implementation of the models and the underlying algorithmic aspects of each. Particularly, we have used TensorFlow, Pytorch and genism for the implementing encoder decoders, transformers and word2vec embeddings respectively. Glove embeddings were available pretrained so all we had to was fetch the vector file from a reputable source. We have also provided a detailed set of test cases, testing all the possible inputs fed to our system and its recorded responses. So far, our system passes all the test cases even after number of testing paths considered.

In chapter 8, we provide a user manual in order to streamline the usability of our system by providing a series of installation, setup and operation instructions.

In chapter 9, we conclude our study by providing a detailed set of experiments conducted to demonstrate our findings and conclude the end results of our tested models. The testing performed has so far been using the rogue score metric. We also plan to incorporate human evaluation as a form of testing.

Table of Contents

Table of Contents i
List of Tables iv
List of Figures v
Chapter 1: Introduction 1
1.1 Purpose of this Document 1
1.2 Intended Audience 1
1.3 Definitions, Acronyms, and Abbreviations 1
Chapter 2: Project Vision 3
2.1 Problem Domain Overview 3
2.2 Problem Statement 3
2.3 Problem Elaboration 3
2.4 Goals and Objectives 4
2.5 Project Scope 4
2.6 Sustainable Development Goal (SDG) 4
Chapter 3: Literature Review 5
3.1 Definitions, Acronyms, and Abbreviations 5
3.2 Detailed Literature Review 5
3.2.1 Encoder-Decoder Model 5
3.2.2 Transformer Model 8
3.2.3 GANS Model 12
3.3 Literature Review Summary Table 14
3.4 Conclusion 16
Chapter 4: Software Requirement Specifications 17
4.1 List of Features 17
4.2 Functional Requirements 17
4.3 Quality Attributes 18
4.4 Non-Functional Requirements 18
4.5 Assumptions 18
4.6 Hardware and Software Requirements 18
4.6.1 Hardware Requirements 19
4.6.2 Software Requirements 19
4.7 Use Cases 19
4.7.1 Signup Form 19
4.7.2 Login Form 20
4.7.3 Generate Summary 20
4.7.4 Store Summary 21
4.7.5 Rate Summary 21
4.8 Graphical User Interface 22
4.9 Database Design 24
4.9.1 ER Diagram 24
4.10 Risk Analysis 25
Chapter 5: Proposed Approach and Methodology 27
5.1 Dataset 27
5.2 Authors of the Dataset: 28
5.3 Statistics 28
5.4 Preprocessing 28
5.5 Word Embeddings 29
5.5.1 Word2Vec 29

5.5.2 Glove 30
5.6 Training Models 30
5.6.1 Encoder Decoder Recurrent neural networks 30
5.6.2 Transformer model 31
5.7 Prediction 32
5.7.1 RNN based models 32
5.7.2 Transformer model 33
5.7.3 Multitask Learning 33
5.8 Evaluation 33
5.8.1 Automatic Evaluation 33
5.8.2 Human Evaluation 34
Chapter 6: High-Level and Low-Level Design 35
6.1 System Overview 35
6.2 Design Considerations 35
6.2.1 Assumptions and Dependencies 35
6.2.2 General Constraints 35
6.2.3 Goals and Guidelines 36
6.2.4 Development Methods 36
6.3 System Architecture 37
6.4 Architectural Strategies 37
6.4.1 Use of Object-Oriented Programming (OOP) paradigm 38
6.4.2 Modular design 38
6.4.3 Scalability 38
6.5 Domain Model/Class Diagram 38
6.6 Sequence Diagrams 38
6.7 Policies and Tactics 42
6.7.1 Coding guidelines and Conventions 42
6.7.2 Testing Strategy 43
6.7.3 Version Control Policies 43
Chapter 7: Implementation and Test Cases 45
7.1 Implementation 45
7.1.1 Encoder-Decoder model 45
7.1.2 Transformers Model 49
7.2 Test case Design and Description 51
7.2.1 Verify Signup with valid credentials. 51
7.2.2 Verify Signup with existing account email. 51
7.2.4 Verify Login with invalid account email and password. 52
7.2.5 Verify get summary by giving text input. 53
7.2.6 Verify save summary privately. 53
7.2.7 Verify save summary publicly. 54
7.2.8 Verify rate summary. 54
7.2.10 Verify change privacy of summary 55
7.2.11 Verify delete summary. 56
7.2.12 Verify show all save summaries (View history of summaries). 56
7.2.13 Verify user can edit profile. 57
7.2.14 Verify forget password recovery. 57
7.3 Test Metrics 58
7.3.1 Sample Test case Matric No.1 58
7.3.2 Sample Test case Matric No.2 58
7.3.3 Sample Test case Matric No.3 58

7.3.4 Sample Test case Matric No.4 59
7.3.5 Sample Test case Matric No.5 59
7.3.6 Sample Test case Matric No.6 59
7.3.7 Sample Test case Matric No.7 60
7.3.8 Sample Test case Matric No.8 60
7.3.9 Sample Test case Matric No.9 60
7.3.10 Sample Test case Matric No.10 60
7.3.11 Sample Test case Matric No. 11 61
7.3.12 Sample Test case Matric No.12 61
7.3.13 Sample Test case Matric No.13 61
7.3.14 Sample Test case Matric No.14 61
Chapter 8: User Manual 63
8.1 Install Node.js on System 63
8.2 Register an account using the Signup Page 63
8.3 Login into your Account 63
8.4 Enter Text 63
8.5 Generate Summary 63
8.6 Save Summary 63
8.7 Rate Summary 63
Chapter 9: Experimental Results and Discussion 65
9.1 Recurrent Neural Networks 65
9.1.1 Simple RNNs with embeddings 65
9.1.2 Attention Based RNNs 66
9.2 Transformers 69
9.3 Human Evaluation 72
9.4 Multitask learning 74
9.4.1 Abstractive-Extractive Model 75
9.4.2 Topic Guided Model 75
9.4.3 Test on CNN/Daily Mail 75
Chapter 10: Conclusion and Future Work 77
10.1 Summary 77
10.2 Conclusion 77
10.3 Future Work 77
References 79

List of Tables

Table 1: Summaries of Research Articles.....14
Table 2: Risk Analysis table.....27
Table 3: Matrix for Glove Model.....32
Table 4: Summary Generated by RNN Models.....67
Table 5: Rouge Scores of RNN Models 68
Table 6: Excessive OOV words generation 68
Table 7: Results without gradient clipping and with gradient clipping 69
Table 8: Summaries Generated by GRU Model 70
Table 9: Rouge Scores of GRU Model 70
Table 10: Summaries Generated on Dataset 70
Table 11: Summaries Generated on cross domain input 71
Table 12: Summaries Generated by RNN Models 71
Table 13: Rouge Scores of Transformer Models 73
Table 14: Rouge Scores of Transformer Models on Technical Dataset 73
Table 15: Summary Generated by SimpleT5 74
Table 16: Sample Summary Generated by Models 74
Table 17: Results Produced by Hybrid Human Feedback 75
Table 18: Results Produced by SimpleT5 Human Feedback 76

List of Figures

Figure 1: Register an account using the signup page.....24
Figure 2: Login into your account.....25
Figure 3: Now paste the desired text in the “Enter News Text” Box.....25
Figure 4: Click on the Summarize Button to generate Summary.....26
Figure 5: ER Diagram.....26
Figure 6: Research Methodology Diagram.....29
Figure 7: Preprocessing Model.....30
Figure 8: Word2Vec Architecture.....32
Figure 9: Encoder-Decoder RNN.....33
Figure 10: Multitasking Model Diagram.....34
Figure 11: Architecture of Transformer Model.....34
Figure 12: Block Diagram about ROUGE Score.....35
Figure 13: Class Diagram of the system.....40
Figure 14: Sequence Diagram for Generate Summary.....41
Figure 15: Sequence Diagram for All Summary List.....41
Figure 16: Sequence Diagram for Login.....42
Figure 17: Sequence Diagram for Set Privacy Status.....42
Figure 18: Sequence Diagram for rate summary.....43
Figure 19: Sequence Diagram for store summary.....43
Figure 20: Sequence Diagram for Signup.....44
Figure 21: Sequence Diagram for summary list.....44
Figure 22: Diagram of Vanilla Encoder-Decoder Model.....47
Figure 23: Diagram of File Structure of Glove Embedding.....48
Figure 24: Diagram of Embedding Layer for Glove.....48
Figure 25: Diagram of Word2Vec Model.....49
Figure 26:Diagram of Language Translation Model.....50
Figure 27: Diagram of Adapted Summarization Model.....50
Figure 28: Diagram of T5 Transformer Architecture.....51

Chapter 1: Introduction

Text summarization is one of the most popular open problems of natural language processing and much of the efforts have already been done in providing coherent and compact representations of large corpuses of text. In particular, the most popular applications of text summarization include text classification, question answering, legal texts summarization, news summarization, and headline generation. However, much of the work done in this field and the tools openly available to a layperson employ extractive summarization techniques, in contrast to abstractive methodology which is the main aim of this project. Extractive summarization is fairly simple and simply involves picking relevant sentences containing much of the semantic information used to express the meaning of a body of text. Much of the work has already been done in this field with popular algorithms such as Text Rank, Lex Rank and even using machine learning methodologies. Abstractive summarization on the other hand not only ensures that the most semantically rich keywords are used in the target summary but also synthetically generated sentences on its own.

1.1 Purpose of this Document

In this document, we will be addressing the overarching problem domain which encompasses our project; namely abstractive text summarization. In particular, this document will describe what exactly abstractive text summarization is and the existing techniques that have been employed to solve this problem primarily using deep learning architectures. We will be discussing the relevance of this particular project, why we chose it, who needs it and what work has already been done in a detailed and comprehensive manner. We will also be describing what the system does, what input it expects and the output it generates. Some light is also shed on what we will be doing and what we won't be. This document also provides a comprehensive literature review on various studies conducted in the field of abstractive text summarization. Three models have been discussed in detail: Encoder Decoder, Transformers and GANS.

1.2 Intended Audience

The target audience of our project are both academia and laypeople who want to understand a large wall of text in a comprehensive manner. This includes people who are too busy to read the entirety of a news article to those who want to understand a complicated research paper without reading every line. Moreover, manual summarization can be very time-consuming. Secondly, journalists too can benefit from an automated summarization tool greatly by producing news headlines as they have to communicate an incident or an event. Giving thorough news is not valuable as compared to quick headliners. So, summarization can help to inform people about daily happenings. Writers too often face the difficulty of creating unique content either blogs or guest posts. They can only produce exceptional content if they know the gist of the whole story. Lastly, creating abstract text summaries not only has the potential to provide compactness but also provide simplification over source text by potentially generating text that simplifies the given text at hand.

1.3 Definitions, Acronyms, and Abbreviations

List of all important acronyms and abbreviations in this document is given below:

SDG: Sustainable Development Goal

GANS: Generative adversarial network

LSTM: Long short term memory

CNN: Convolutional Neural Networks

Conclusively, in chapter 1, we have discussed the overarching motive of this project and the importance of text summarization in different spheres of academia, research and most educational domains. In the next chapter, we will be relating the problem introduced to both its research and development implications by discussing the domain our problem belongs to and elaborating a birds eye view of this problem's specifications. Lastly, in the last chapter of this document, we present a brief survey of past studies in relation to the objectives we are trying to achieve.

Chapter 2: Project Vision

2.1 Problem Domain Overview

Our problem belongs to the overarching domain of Neural Machine Translation or language modeling which is the problem of translating one sequence to another of different lengths using neural network models or deep learning architectures. This also involves text summarization which is the end goal of this project. Our project performs this summarization task using deep learning models primarily with the help of sequence to sequence models. Our problem can also be seen as variation of language translation systems where the job of the translator is to map one sequence belonging to one language to another having different length from the source text. Except that in this case, the languages are the same but similar to language translation the wording and length of the target text is different.

The primary goal of developing this system is to take a plain text document as input and generate a summary in its own words rather than copy pasting a select few sentences from the source document. The summary shall retain the semantics of the original document while providing a few lines of compact text representation of the input text. Testing shall be done on different neural network models and results shall be compared. These baseline models' results will be compared using some evaluation methods and determine the one that works best for our task. After performing initial testing, we will be exploring if any further improvements can be made and introduce improvements of our own to the existing baseline models and adapt it to a suitable application for demonstrating our results.

2.2 Problem Statement

Our summarization tool should aim to generate compact, relevant, factually consistent, grammatically coherent and human readable summaries.

2.3 Problem Elaboration

Our project can be divided into following high level components with the following problems to be solved:

1. **Dataset collection:** In this task, we shall find the dataset that has been tested numerous times on various models such that it is easy to benchmark our evaluation and accuracy against existing methodologies.
2. **Text pre-processing:** Text shall be cleansed and treated according to our task. Which typically involves the following major high level steps: noise removal, disambiguation of words and tokenization.
3. **Vectorisation:** The cleaned text shall be converted into numerical representations understood by computers, these are fixed length vectors or also known as word embeddings.
4. **Model development:** Appropriate deep learning models shall be developed such as Encoder Decoder or transformers, they shall be trained and tested on appropriate splits of train and validation test sets.
5. **Evaluation:** Finally, our model's generated summaries shall be compared with target summaries using an evaluation metric: Rogue scores, BLEU or METEOR.

2.4 Goals and Objectives

In this study, we want to accomplish the following milestones:

1. Explore different summarisation datasets and see which helps us improve our architectures to produce as human like summaries as possible.
2. Explore different evaluation methods that can be used for evaluating abstractive summaries: ROGUE scores, BLEU scores and METEOR.
3. Explore different word representations and see if they have any impact on the scores or accuracies for summarisation tasks.
4. Explore various baseline models that can be used for summarisation and see if any improvements can be made on these architectures and introduce some innovations of our own.

2.5 Project Scope

1. We will only focus on abstractive text summarization.
2. We will only focus on summarization of text that falls under current affairs.
3. We will only focus on English text summarizations. Other languages are not within the scope of the project.

2.6 Sustainable Development Goal (SDG)

The sustainable development goals (SDG) that this project will target is quality education, since this SDG goal aims to provide children and young people with quality and easy access to education plus other learning opportunities. Abstractive text summarization achieves two goals in particular:

1. Compact summary
2. Wording different from the source

Which ensures that not only the source text has become lesser in word length but is also more accessible to its end users, this feature is especially relevant for people belonging to academia who want to understand rigorous scientific material in bulk and be able to review them. This ensures greater impairment of knowledge and encouragement of critical thinking, allowing students with less rigorous technical background to get awareness in fields outside their expertise.

In other words, our summarization goal will help information become more accessible to people with varying backgrounds for literacy in subject areas beyond their expertise and formal qualifications. Even for lay people, our summarization goal imparts awareness in the current state of affairs. News summarization allows people busy in their jobs to be able to quickly gloss over a shortened and more simplified wall of text to be able to get a good idea of the current affairs.

Chapter 3: Literature Review

3.1 Definitions, Acronyms, and Abbreviations

List of all important acronyms and abbreviations in the literature review is given below:

RNN: Recurrent Neural Network

BERT: Bidirectional Encoder Representations from Transformers

ROUGE: Recall-Oriented Understudy for Gusting Evaluation

CNN: Convolutional Neural Networks

Seq2Seq: Sequence to Sequence

LSTM: Long Short-Term Memory

T5: Text to Text Transfer Transformer

R1: ROUGE-1 Score

GRU: Gated Recurrent Unit

BLEU: Bilingual Evaluation Understudy

OOV: Out of Vocabulary

C-GAN: Conditional-Generative Adversial Network

3.2 Detailed Literature Review

This section will include detailed literature review of your problem area. Make different categories for different types of work done in the past. In addition to textual descriptions, make a summary table that describes each paper that you have read, along with references.

3.2.1 Encoder-Decoder Model

Abstractive method of text summarization using Seq2Seq RNNs

Summary

This research uses LSTM units with encoder-decoder architecture, using typical attention mechanisms in the decoder layer. A typical decoder only uses one context vector, outputted from the encoder and uses it to create a list of tokens in a recurrent fashion. However, the attention mechanism not only takes this context vector but also concatenates the hidden states of the encoder units. It learns "attention weights" that lets it take into account specific intermediate states, required to produce a better decoded output. The dataset used in this research is amazon fine food reviews dataset that uses food descriptions to generate a summary.

Critical Analysis

Compared to previous studies, this research has managed to increase the accuracy by 3.6%, but some issues still need to be addressed. Even though by design, this model is robust towards working on large sentences, proper evaluation metrics are needed such as ROGUE scores and BLEU metrics to see how well they perform against existing baseline models. Furthermore,

Amazon fine food reviews itself doesn't contain any long sentences nor long summaries, so we can't say for sure how well this model does on datasets with larger text sequences.

Relation with the Proposed Work

Our research work involves exploration of abstractive text summarization techniques using deep learning models and since, LSTM encoder decoder is also a deep learning architecture built on top of RNNs, encoder decoder model using attention mechanism is directly related to our focal point of research.

Indonesian Abstractive Text Summarization Using Bidirectional Gated Recurrent Unit

Summary

This paper too uses sequence to sequence RNNs based architecture, except that it proposes to replace LSTM units with GRU units. The dataset used for this experiment were Indonesian journal documents with 500 documents; 400 used as training and 100 used as test set. The model proposed by this research produces Rogue 1 score of 11% and Rogue 2 score of 1.1%. According to the experimental results, the model was able to generate abstracts with relevant pieces of facts and right keywords but, however, wasn't able to produce coherent sentences free of grammatical shortcomings. The reason pointed out for this was apparently the long text sequences used as source documents. And to avoid any loss of information from the documents, their lengths weren't trimmed down either.

Critical Analysis

Even though the model produced fairly poor rogue scores, it helps us expose the problems associated with vanilla encoder decoder architectures when they are applied on documents with long texts. Unlike amazon fine food reviews, journal abstracts and documents are huge in length and hence, prone to long sentence problems faced by sequence to sequence architectures. Secondly, usage of GRU instead of LSTM guarantees a simpler model to train thereby reducing training overhead and degradation of performance; allowing faster convergence.

Relation with the Proposed Work

Like previous studies we have already presented, this model is a variation of encoder decoder model which are a famous deep learning architecture and part of our study for various deep learning existing architectures. Rogue scores presented in this study are standard metrics used for summary evaluation which are likely to be incorporated in our research.

Abstractive text summarization using LSTM-CNN based deep learning

Summary

This paper presents a novel approach to use CNN and LSTMs for text summarization. The dataset used for this project was CNN/Daily Mail and claims to outperform previous state of the art models. Words in a sentence are converted into vectors using word2vec and stacked in a 2 Dimensional matrix fed to the CNN encoder. Two convolutional filters/kernels are used: one to extract Relational Phrases and one for Object/Entity phrases. The two are max pooled and their results are combined using sum, concatenation or average into one encoded vector which is fed to the usual LSTM encoder decoder model to generate a summary. The experiment shows that this technique achieves ROUGE-1 score of 34.9% and the ROUGE-2 of 17.8%.

Critical Analysis

This study has managed to obtain fairly adequate rogue scores compared to our previous studies. Which means that this model does a better job of producing sentences with grammatical cohesion and consistency. However, in this technique, the model generated summary involves many redundant and repetitive words which can be improved by incorporating the attention mechanism used in previous models since attention makes sure to learn attention weights for previous and next contextual information in the decoder while generating a new sequence.

Relation with the Proposed Work

This study uses CNN and LSTM encoder decoder architectures as the main components of the system which primarily are classical deep learning models used for a variety of deep learning problems related to text, speech and images. Since our primary focus of research are deep learning models for abstractive text summarization, This research provides us with more horizons to research and experiment.

A hierarchical Seq2Seq Attention model with Copynet for summarization

Summary

This paper also uses the typical Encoder Decoder model but with two major enhancements: Hierarchical Attention and Copynet to solve two prevalent problems in most NLP systems: Missing words problem and sentence level contextual richness. This model too has been tested using the ROGUE score metric, producing ROUGE 1 Score: 36.11% and ROUGE 2 Score: 18.37%.

Hierarchical Attention mechanism is fairly intuitive. Just like how words and their activations from encoder units in the LSTM were combined, this model concatenates context vectors generated by multiple LSTM units and performs an attention operation(addition) on many of such phrase level combined vectors. The result of this operation is fed to the decoder alongside other typical inputs expected. Copy mechanism is the most interesting aspect of this paper. The decoder layer has two modes where it learns to choose between a typical word generation SoftMax layer(basically the vocabulary shortlist vector) and a pointer SoftMax layer(the location of the copy token in the input document). The decoder consists of a gate which learns to switch between generation and copying mode using sigmoid function.

Critical Analysis

One significant advantage provided by this study is the incorporation of sentence level attention that can definitely introduce more contextual richness. Second important contribution is the Copying mechanism that solves the rare words or missing word problem by directly copying a word/token from the input rather than generating it if it's missing. Due to these added enhancements, the model was able to outperform the above presented CNN LSTM model in rogue scores.

Relation with the Proposed Work

Uses neural networks deep learning models in both copynet and encoder decoder architecture which are directly related to our primary research of deep learning models for abstractive text summarization. Copynet in particular solves OOV(out of vocabulary) words problem which is an existing complication in most deep neural machine translation tasks, which can be helpful in our study.

Abstractive and Extractive Text Summarization using Document Context Vector and Recurrent Neural Networks

Summary

In this paper, the author presents a novel approach to perform both abstractive and extractive summarization using a document context vector. The setting presented by this study are ecommerce websites however, the outcomes of this study can be applied in most other identical settings given enough information regarding documents are available(which they usually are). The dataset/data creation process of this study involved human extracted summaries from eBay (title, URL, description) were used as the metadata described below plus the actual reviews.

The document vector presented in this study is analogous to abstract and title of a document. In a nutshell, without going into the mathematical rigor, the vector is a summation of these three vectors: Query Vector(what user looks up to search for the document),Browse Vector(what document is opened up for this query) and Document Creator Information(Meta data, product specification). This is pretty similar to a decoder receiving a context vector from an encoder, necessary for generating the decoded context vectors. Except that this document context vector helps in a good encoding scheme using a similar concept. The results reported by this model has Rogue 1 score of 0.45 and Rogue 2 score of 0.31.

Critical Analysis

This study has managed to outperform all the models presented so far with very high both rogue 1 and rogue 2 scores. The high rogue 1 score is attributed to inference of semantic meaning of the document and therefore generating highly relevant summaries. R2 score is more associated with high readability.

The incorporation of context vector allowed this model to have high inference ability and context capturing at the same time which hasn't been achieved in any of our previous studies presented so far, which can be attributed to a good initialization of hidden state: in this case, being a semantically rich context vector. This study however is very specific to ecommerce websites and isn't highly extensive for other more general purpose text summarization tasks. Furthermore, collection of data required for browse and query vectors isn't very easy to obtain and acquisition of its complexity depends on user behavior and product demand which in itself isn't very extendible to other novel tasks of text summarization.

Relation with the Proposed Work

In this study, the author has presented deep learning models known as encoder decoder architecture with a specialized hidden state initialization at time step t0 which is an essential step for training deep learning models and hence, relevant to our work of training deep learning models for the task of text summarization. An alternating initialization strategy has been presented but since we will be focusing on other datasets not in the ecommerce domain so it won't be highly suited to our task. Furthermore, the author also presents a technique for generating data for extractive summarization which is beyond the scope of our study.

3.2.2 Transformer Model

Automated News Summarization Using Transformers

Summary

The authors of this research paper have used the datasets from BBC news. In the method of abstractive summarization, an interpretation is first created by analyzing the text document. Based on this interpretation, the machine predicts a summary. It transforms the text by

paraphrasing sections of the original document. For preprocessing of the dataset, different techniques were used such as lower casing, eliminating punctuation, eliminating frequently occurring words, stemming (reducing the infected words to their root form), lemmatization (reducing derived words to their base form) and contraction mapping (expanding the shortened version of words).

In this paper, the authors have used the state of the art T5 transformer model. The T5 model which stands for "Text-to-Text Transfer Transformer". It basically works on the concept of transfer learning. The model encodes the input text via cross attention layers and then feeds it to decoders. The decoder then generates autoregressive output.

Critical Analysis

The authors have compared different pre-trained language models for the task of text summarization such as Pipeline Model, BART model, T5 model and PEGASUS model. They came to know that the T5 outperformed all other models. For future work, a hybrid of the above-mentioned models can be used to create more meaningful summaries and improve overall accuracy. More robust models can be built by extending this algorithm. This model can also be applied to multi-document summarization and for producing summaries of variable length in future.

Relation with the Proposed Work

As our research work is related to abstractive text summarization using various state of the art deep learning models, the T5 transformer model used in this research paper also has a deep learning architecture using encoders and decoders, therefore, it is directly related to our focal point of research.

Evaluation of the Transformer Model for Abstractive Text Summarization

Summary

The datasets for this research were taken from a CNN/Daily Mail article. The authors have implemented both the models with Tensor2tensor library. The transformer model uses only the attention mechanism in combination with feed forward networks. The decoder and the encoder segments have the same structure but an additional Multi-Head attention layer known as Masked Multi-Head attention layer is added in the decoder segment. The authors have also added positional encoding to the input in-order to keep track of the input sequence. When the results of the transformer model and seq2seq model were compared, it was found that the transformer model performs 28 to 45 percent better than seq2seq model. It was also found that the transformer model tends to generate shorter sequences as compared to the seq2seq model. The transformer model trains faster and can even train on long inputs and output sequences as compared to seq2seq model.

Critical Analysis

The authors of this research paper have compared the transformer model with LSTM-based encoder-decoder model with main focus on abstractive text summarization. The results of this research paper shows that the transformer model produces better results and summarization as compared to the LSTM based model. However, the model used in this research paper has some drawbacks as well. Hyper-parameter tuning for the model has been limited as the model required a lot of training time. Since the datasets of CNN/Daily mail articles only have one reference, therefore, evaluating summary content is challenging sometimes.

Relation with the Proposed Work

As the transformer model used in this research paper has achieved higher average ROUGE scores as compared to other models, therefore this model can widely be used in our research work for abstractive text summarization using deep learning models. As this model also seems to be less affected by variable input length, therefore, it can be used for news articles as well. The model can also train on longer input and output sequences while consuming less memory, therefore, it will save our time as well during research.

Sample Efficient Text Summarization Using a Single Pre-Trained Transformer

Summary

The models consist of a transformer encoder network that reads the article, a transformer decoder model for generating summary and an encoder-decoder attention mechanism. This mechanism allows the decoder to attend to the encoder states during generation. The authors have concluded that using a single pre-trained transformer model for sequence to sequence task makes text summarization easy and precise. It reduces the overall number of parameters and it also removes the non-retrained encoder-decoder attention weights. The results show that having every parameter being pre-trained instead of only a single subset of them provides large gains in limited data.

The authors of this research paper have used the datasets from CNN/Daily Mail articles. The experiments on these datasets show that the pre-trained Transformer model has more efficiency than other encoder-decoder models by over 10 ROUGE- 2 points.

Critical Analysis

Instead of using separate encoder and decoder models, the authors have used a single component for both encoding the source and generating the target. This approach helps to ensure that no copies of data are produced and less parameters are used for encoding and decoding. Fine tuning the model on only 1 percent of data produces a ROGUE-2 score of 12.1 whereas, an encoder-decoder model only produces 2.3 ROUGE-2 score. This shows that the transformer model used in this research paper has outperformed other encoder-decoder based models in terms of ROUGE scores.

Relation with the Proposed Work

As the pre-trained transformer model used in this research paper shows relatively good ROUGE scores as compared to other encoder-decoder models, therefore, we can use this model for our research work in abstractive text summarization using deep learning models. This model is highly related to our proposed work as it is trained on CNN/ Daily Mail news articles and the datasets of our research work is also based upon news articles.

A Two-Stage Transformer-Based Approach for Variable-Length Abstractive Summarization

Summary

This research paper proposes a two stage transformer based method for variable length summarization. This model consists of two parts, a text segmentation module and a two-stage transformer based summarization module. The role of the text segmentation module is to divide the input text into different segments using pre-trained bidirectional encoder representation from Transformers (BERT) and bidirectional long short-term memory (LSTM). The most important information is fetched from the segments using the BERTSUM model. After fetching important information, training of the two-stage summarization model is performed.

For training, the important information fetched using BERTSUM model is utilized to train the document summarization module. In the next step of training, this trained data and important information are used to train the segment summarization module. Then, on the basis of loss score of segment summarization and document summarization module, parameters of segment summarization module are updated. At last, collaborative training is applied on segment summarization and document summarization modules. By this procedure, 70 percent accuracy in human subjective evaluation was achieved.

This framework is divided into two phases which are the training phase and testing phase. The testing phase is further divided into four basic models which are text segmentation model, the extractive model, the document transformer model and the segment transformer model. In the training phase, firstly a BERTSUM based model is trained.

Critical Analysis

The model achieved better or comparable performance in both objective and subjective evaluations as compared to other advanced models discussed in previous papers. However, this model used in this research paper has some drawbacks as well. The size of information is insufficient for the variable length summary. Most of the articles are short, due to which this method fails to derive a good variable length summary. Therefore, short articles will result in poor quality of summary. This model also fails to detect the size of information present in the article.

Relation with the Proposed Work

In this model a state of the art deep learning model known as BERT (bidirectional encoder representation from Transformers) is used which has achieved better or comparable accuracy as compared to other deep learning models. Therefore, this model can be used in our research work which focuses on producing abstractive text summaries for news articles by using deep learning models. This model also has separate training and testing phases which will help us to evaluate and train our model more efficiently.

Transformer-based Model for Single Documents Neural Summarization

Summary

The authors of this article have used datasets from CNN/Daily Mail and News- room for this research. They have used the encoder-decoder model but with an extra focus on the encoding phase. For this purpose, they have introduced the encode-encode-decode model. This model first encodes the source text by the help of a transformer model and then applies sequence to sequence model for another encoding layer. By this method, authors came to know that Encoder Decoder model and Transformer model complements each other which converts the source text into richer encoded vector representation. They have also discovered that during abstractions, focusing on important vocabulary words improves the performance of this model.

Critical Analysis

The transformer model used in the research paper is simple to train the intuition behind the formulation of this model is logical, straightforward and easy to understand. However, the model used in this research paper has a drawback as well. Abstractive summarization was not able to reproduce the salient parts of the given document. However, it rewrites them in a concise form, normally introducing novel words by utilizing some key abstraction techniques such as paraphrasing, compression or sentence fusion. Therefore, it was able to meet challenges like grammatical correctness and repetition of words in long sentences.

Relation with the Proposed Work

As the model used in this research paper complements other encoder-decoder models, therefore we can use this model as a deep learning technique for our research work which mainly focuses on abstractive text summarization using deep learning techniques. This model is easy to train and the logic behind this model is easy to understand, therefore, making changes in this model will be easy and less time consuming. Therefore, by using this model as our basic deep learning technique for our proposed work, we can make amendments in the model as well, thereby improving the overall accuracy of this model.

3.2.3 GANS Model

Transformer-based Model for Single Documents Neural Summarization

Summary

It is using semantic distance and fine turning with decoder summary. It is implementing GAN's as encoder decoder with adversarial training. Usually, GAN summary generation is conditional probability mapping but here it is using new architecture which combines encoder decoder with adversarial network. Introduced additional encoder-decoder model for summary encoder pre-training and discriminator subnet for enhancing the original text. The encoder decoder can choose CNN, RNN or transformer network. For seq2seq it achieves up of 3.53 for ROUGE-1, 3.30 for ROUGE-02 and 5.91 for ROUGE-L respectively.

Critical Analysis

Its uniqueness is it introduced distance between the original text representation and standard summary representation into encoding stage as model optimization goal through adversarial training. It learns and optimizes the continuous encoding distribution source text rather then word by word model optimization procedure. It eases the discreate data optimization and conditional generation problems in traditional GAN model. To alleviate the problem of insufficient and inaccuracy attention to important information of original text when generating summary, it introduced keyword information when encoding the input sequence.

Relation with the Proposed Work

It is using a GAN model and within it is using an encoder decoder. Our and this problem both fallen seq2seq problems. It is using CNN and RNN which is a major focus point of our research. The rough score will also be standard used by us. All underline deep learning models it is using are our research area for creating deep learning models for abstractive text summarization.

Transformer-based Model for Single Documents Neural Summarization

Summary

The model uses a cGAN like architecture where the generator and discriminator both take a type and source text as input, to make the model generate either a headline or summary according to user requirements, which is indeed a novelty in terms of utility provided by text summarization tasks, allowing us to generate both summaries and titles of articles at the same time. The generator is seq2seq model and discriminator is binary classifier consist of LSTM layers. It uses Cornell newsroom which is 1.3 million articles from 38 news publishers. The model gives scores of R1: 29.51, R2: 6.43 and RL: 22.35 for summary generation task and R1: 15.64, R2: 1.64 and RL: 12.47 for headline generation.

Critical Analysis

This methodology too uses unsupervised learning using near identical optimization objectives of reconstruction and readability. Except that it not only uses summaries but also feeds headlines to the discriminator in a joint manner, therefore providing a diverse data set to the model. Even though the model didn't manage to outperform, it still managed to adequately produce both summaries and headlines in a reasonable manner Even though it didn't manage to outperform the previous studies, it still managed to provide other major milestones as unsupervised learning and joint learning allowing us to produce semantically relevant texts of different nature for same document. Even though the model performs poorly in modelling R2 dependencies but provides decent valuation scores for the other metrics: Indicating both relevance and grammatical consistency.

Relation with the Proposed Work

Our research is on using deep learning models for abstractive text summarization. Here it is using the seq2seq model on which our problem falls. Use discriminator which is LSTM based which is a deep learning model which is our research domain for creation of abstractive text summarization.

Transformer-based Model for Single Documents Neural Summarization

Summary

By extracting triples as keywords from the summary which is used by generator to generate summary. They use different training strategy pretrain generator using given meta data then use sample summary generated by generator and real summary to pre-train the discriminator. The generator is seq2seq model which is composed of encoder decoder. The encoder uses a 128-dimensional bi-directional LSTM which transform the input to fixed length background vector and decoder receive this vector and generate summary. The discriminator is binary classifier which is convolutional neural network using three filters 3,4 and 5 dimensions and hash three layers convolutional, max-poling and SoftMax. It uses CNN/Daily Mail dataset 286,817 training pairs, 13368 verification pairs and 11,487 test pairs. It gives results of 38.07, 14.98, 34.80 and 0.767 for ROUGE-1, ROUGE-2, ROUGE-L and Time respectively.

Critical Analysis

Different from standard training strategy it uses additional teacher mode which is entity relation extraction network. Model has 0l.45 improvement on ROUGE-1 compared to baseline and pretraining time has been shortened by 23% relatively.

Relation with the Proposed Work

Seq2seq, encoder decoder and LSTM all come under deep learning and here it is using all of them to generate abstract summaries. This will help us to learn and implement our model which is based on the seq2seq problem and using the encoder decoder paradigm. The rough score is the standard of showing results used by this paper and part of our model results presentation.

Transformer-based Model for Single Documents Neural Summarization

Summary

Conditional GANs were developed to control the class label of the data generated by G. This study proposed CGAN for summarizing bio-medical information. This model basically consists of a conditional GAN feature matrix. The model works by first passing the document feature matrix by the help of a CNN network. Then, this information is further passed to a connected layer and then to the output layer. The probability of importance of a sentence in the

article is determined by the output value, which is a number between zero and one, where one signifies that the sentence is present in the summary. The layer of discriminator is a neuron. In this study, 500 medical articles were taken from PubMed Central to evaluate the model. For both discriminator and generator, two-dimensional convolution is used. It gives result of 43.78 and 26.73 for rough-1 and rouge-2 respectively.

Critical Analysis

Different experiments on the medical datasets and this method recorded an overall 5% increase in accuracy than all 8 other competing methods. The summary produced was also like the original reference summaries. The purposed summarizer is compared with eight methods. These methods include various approaches, such as statistical, graph-based, and deep learning.

Relation with the Proposed Work

Our research work involves exploration of abstractive text summarization techniques using deep learning models and it is using GAN and CNNs since GAN is a deep learning model and within GAN the generator and discriminator any neural network can be used here it is using CNN which is a deep neural network with multiple hidden states. This will help us to build our model using CNNs deep neural networks.

3.3 Literature Review Summary Table

Table 1: Summaries of Research Articles

The summary of all research articles used in the literature review is presented here.

No.	Name of Research Paper, Reference	Year of Publication	Result	Description
1.	Abstractive method of text summarization using Seq2Seq RNNs, [1]	2019	3.6% increase in accuracy	This research uses LSTM units with encoder-decoder architecture, using typical attention mechanisms in the decoder layer.
2.	Indonesian Abstractive Text Summarization Using Bidirectional Gated Recurrent Unit , [2]	2019	R1 score of 11% and R2 score of 1.1%	This paper also uses sequence to sequence RNNs based architecture, except that it proposes to replace LSTM units with GRU units.
3.	Abstractive text summarization using LSTM-CNN based deep learning, [3]	2019	R1 score of 34.9% and R2 score of 17.8%	This paper presents a novel approach to use CNN and LSTMs for text summarization.
4.	A hierarchical Seq2Seq Attention model with CopyNet for summarization, [4]	2018	R1 score of 35.11% and R2 score of 18.37%	It uses the typical Encoder Decoder model but with two major enhancements: Hierarchical Attention and CopyNet to solve missing words problem and sentence level contextual richness.

5.	Abstractive and Extractive Text Summarization using Document Context Vector and Recurrent Neural Networks, [5]	2018	R1 score of 45% and R2 score of 31%	It presents a novel approach to perform both abstractive and extractive summarization using a document context vector.
6.	Automated News Summarization using Transformers, [6]	2022	R1 score of 47% and R2 score of 33%	State of the art T5 (Text-to-Text Transfer Transformer) transformer model is used in this paper which works on the concept of transfer learning.
7.	Evaluation of the Transformer Model for Abstractive Text Summarization, [7]	2019	R1 score of 39.06% and R2 score of 16.76%	It has attention mechanism in combination with feed forward networks. Both models are implemented with Tensor2tensor library.
8.	Sample Efficient Text Summarization Using a Single Pre-Trained Transformer, [8]	2019	R1 score of 12.1% and R2 score of 2.3%	The models consist of a transformer encoder network that reads the article, a transformer decoder model for generating summary and an encoder-decoder attention mechanism.
9.	A Two-Stage Transformer-Based Approach for Variable-Length Abstractive Summarization, [9]	2020	R1 score of 37% and R2 score of 17.86%	It has two stage transformer based method for variable length summarization. It has a text segmentation module and a two-stage transformer based summarization module.
10.	Transformer-based Model for Single Documents Neural Summarization Summary of the research item, [10]	2019	R1 score of 41.89% and R2 score of 18.9%	It has encoder-decoder model but with an extra focus on the encoding phase. The model is known as encode-encode-decode model.
11.	A Generative Adversarial Constraint Encoder-Decoder Model for the Text Summarization, [11]	2022	R1 score of 38.31% and R2 score of 19.89%	It is using semantic distance and fine turning with decoder summary. It is implementing GAN's as encoder decoder with adversarial training.
12.	Generative Adversarial Network for Joint Headline and Summary Generation, [12]	2019	R1 score of 3.3% and R2 score of 5.91%	The model uses a C-GAN like architecture where the generator and discriminator both take a type and source text as input to make the model generate summary.
13.	Abstractive Text Summarization	2021	R1 score of 15.64% and	It uses different training strategy pretrained generator and then use

	Using Generative Adversarial Network and Relation Extraction, [13]		R2 score of 1.64%	sample summary generated by generator and real summary to pre-train the discriminator.
14.	Biomedical text summarization using Conditional Generative Adversarial Network (CGAN), [14]	2021	R1 score of 38.07% and R2 score of 14.98%	Conditional GANs were developed to control the class label of the data generated by GANs. This study proposed CGAN for summarizing bio-medical information.

3.4 Conclusion

In this paper, we have managed to present a comprehensive literature review of some existing state of the art deep learning techniques used for text summarization. Among which some have advantages over the other and give us a waypoint for future research into comparing these models and use the most appropriate one for text summarization. Even though inherently, sequential models can give more accurate results, GANs with some tuning and their greater performance potential, can yield some interesting benefits of their own. One of them being spatial temporal dependence. Meaning, like an image, it can extract valuable features out of a text matrix and recognize important contexts which is essentially what attention mechanisms are too are good at. However, the downside to this approach are obviously memory limitations and usage of excessive memory pools and large matrices used as convolution layers used for large number of feature filters.

Chapter 4: Software Requirement Specifications

The software requirement specification is for the website that we have made for the abstractive text summarization using deep learning tools. Following are the important items covering the software requirements:

4.1 List of Features

Following are the important features of our website:

1. Paste Text from any news article or website
2. Generate Summary using Multiple Model Feature with Rouge Scores
3. Login and Signup Feature
4. Save the summary feature
5. Seeing the history of summaries feature
6. Rating the summary feature
7. Changing privacy status of the summary feature
8. Searching the summaries feature

4.2 Functional Requirements

1. User shall be able to sign up:

This has high priority. User will be able to sign up by providing name, email and password.

2. User shall be able to login/logout.

This also has high priority. User will be able to login by providing their email and password.

3. User shall be able to edit his profile.

This has low priority. User shall be able to change their name and email. They shall be able to change basic information written in 'My Profile' page.

4. User shall be able to reset his password if he forgets password.

This has low priority. User shall be able to reset their password if forgotten using SMS or email.

5. System shall provide a summary for input text given by user.

This has high priority. The system shall generate summary of the input text.

6. User shall be able to select one model out of multiple summarization models.

This has high priority. System shall allow the user to select one model out of multiple models available.

7. System shall generate Rouge scores for the generated summary.

This has high priority. User shall be able to see the rouge score as well of the summary.

8. User shall be able to save the summary.

This has medium priority. System shall be able to save the generated summary.

9. User shall be able to view history of summaries.

This has medium priority. System shall be able to see the history of saved summaries.

10. User shall be able to rate the summaries.

This has medium priority. User can rate summaries from one to five star. This data can be helpful to train the model properly and find any errors in the summary.

11. User shall be able to change the privacy status of the summaries.

This has low priority. This shall enable other users to see the summaries generated by other users and can help them understand the topic much better.

12. User shall be able to search summaries.

This has low priority. This shall enable the users to save their time and read only important summaries which they need for their research/knowledge.

4.3 Quality Attributes

Following are the quality attributes of the SRS document:

1. Signup
2. Login
3. Generate Summary
4. Store Summary
5. Rate Summary

4.4 Non-Functional Requirements

Following are the important non-functional requirements covering our website:

1. **Performance**: Thousands of users shall be able to access the system at the same time.
2. **Reliability**: The website shall be available 24 hours a day. However, it might go offline to import retrained model.
3. **Reusability**: Intuitive interface shall be provided to the users.
4. **Security**: The website shall be able to provide privacy of data to each user.
5. **Supportability**: The system shall be updated frequently.

4.5 Assumptions

We have made following assumptions for the software requirement specification:

1. We have assumed that the user has the knowledge of how to copy the text from any news article from the website or document.
2. We have assumed that the user shall only input news articles in the form of text, and not any kind of code or mathematical expression as our model can't recognize its importance.
3. We have assumed that the user shall only summarize English news articles as our model is trained only on English language.

4.6 Hardware and Software Requirements

Following are the details about the hardware and software requirements for our website.

4.6.1 Hardware Requirements

Only Laptop/PC or any other device which has browser extension is required. There is no need of any special hardware for the users of the website.

4.6.2 Software Requirements

The application will be developed using REST framework API and Django. Other than that python and TensorFlow for backend models, React for frontend and SQLite for database will be required. Additionally, other utilities will be needed including:

1. VS code text editor
2. Postman API Platform
3. Git version control
4. Node development server
5. Transformer, Rouge score and Pytorch Library installation using pip.

4.7 Use Cases

Following are five main use cases for our website:

4.7.1 Signup Form

Name	Signup Form		
Actors	Admin, Client		
Summary	The user shall provide their email, name and password on the signup form and after successful verification, redirect the user to the login page.		
Pre-Conditions	None		
Post-Conditions	The user's profile is successfully created and shall be redirected to the login page.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	The user navigates to sign up page.	2	The sign up page is displayed asking for email, username and password.
3	The user enters the email, username and password and clicks "sign up" button.	4	The System shall create a new user account and redirects the user to the login page.
Alternative Flow			
3	The user enters invalid email or password.	4-A	The system responds with an error message: *Incorrect email or password entered.*

4.7.2 Login Form

Name	Login Form		
Actors	Admin, Client		
Summary	The user shall provide their email and password on the login form and after successful verification, redirect the user to the their dashboard.		
Pre-Conditions	User has already registered an account.		
Post-Conditions	The user's session is successfully established and shall be redirected to their dashboard.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	The user navigates to login up page.	2	The login page is displayed asking for email and password.
3	The user enters the email and password and clicks "login" button.	4	The system verifies email and password, establish a session for the user, and redirects the user to their dashboard.
Alternative Flow			
3	The user enters invalid email or password.	4-A	The system responds with an error message: *Incorrect email or password entered.*

4.7.3 Generate Summary

Name	Generate Summary		
Actors	Admin, Client		
Summary	The user shall be able to generate a summary of an article or text entered.		
Pre-Conditions	User is logged into the system and the user is on summary generation page.		
Post-Conditions	System generates a summary for the provided article.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	The user navigates to the summary generation page.	4	System generates a summary of the article and displays it to the user.
2	User pastes/types the article.		
3	User selects the model and click on "Summarize" button.		
Alternative Flow			

2	The user paste an article which can't be retrieved or parsed.	4-A	The system responds with an error message and asks the user to try another article.

4.7.4 Store Summary

Name	Store Summary		
Actors	Admin, Client		
Summary	The user shall be able to store the summary of an article or text entered.		
Pre-Conditions	User is logged into the system and the user is on summary generation page.		
Post-Conditions	The generated summary is saved in the user's account.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	User clicks the "Save Summary" button on the summary page.	2	System saves the summary to the user's account.
Alternative Flow			
2	The user has already saved the summary.	2-A	The system responds with an error message and does not save the summary.

4.7.5 Rate Summary

Name	Rate Summary		
Actors	Admin, Client		
Summary	The user shall be able to rate the summary of an article or text entered.		
Pre-Conditions	The user is logged into the system. User has generated at least one summary and the summary being rated exist and is accessible by the user.		
Post-Conditions	The user's rating is recorded for the summary. The average rating for the summary is updated in the system. The user is provided with a confirmation message indicating their rating has been recorded.		
Special - Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	The user navigates to the summary they want to rate.	3	The system shall provide the user with a confirmation message indicating their rating has been recorded.

2	The user selects a rating between 1 and 5 stars.		
Alternative Flow			
2	User attempts to rate a summary they have already rated.	3-A	The system notifies the user that they have already rated the summary and does not record the new rating.
2	User attempts to rate a summary that does not exist or is not accessible by the user.	3-A	The system displays an error message indicating that the summary cannot be rated.

4.8 Graphical User Interface

This section should give the GUI dumps of each screen, with reference to the users. The navigation flow of each user is also required, and each GUI should mark the functionality/use case that it covers.

Following is the list of GUI images which helps the user to understand how to generate the summary.

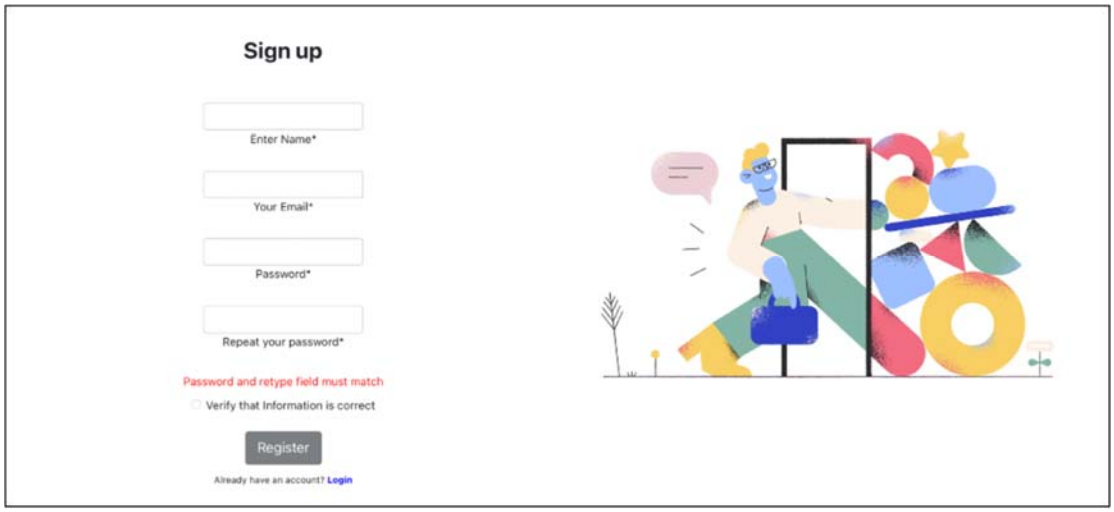


Figure 1: Register an account using the signup page.
The above figure explains the first step involved in generating the summary on the website.

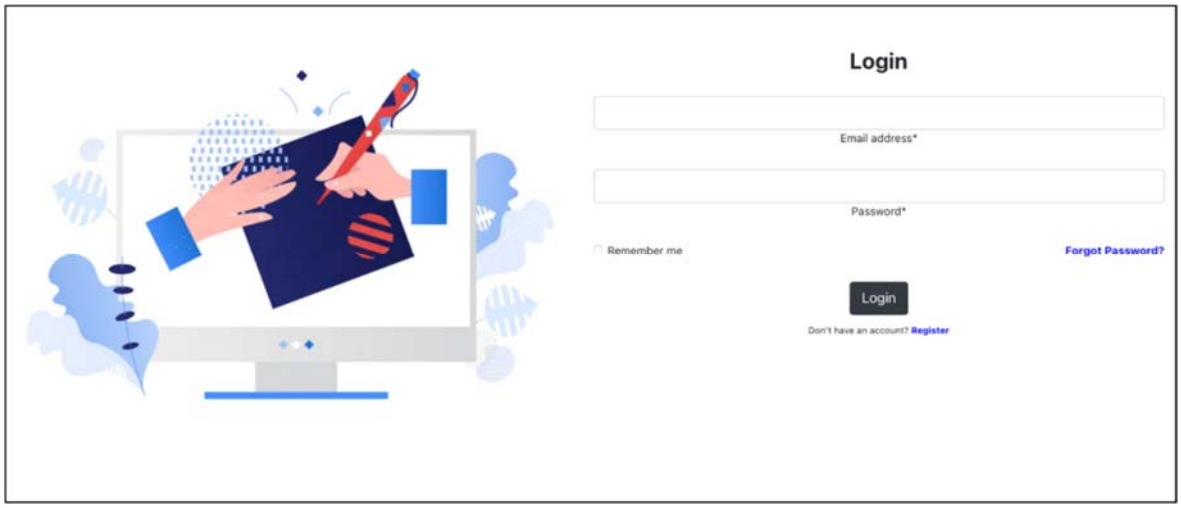


Figure 2: Login into your account
The above figure explains the second step involved in generating the summary on the website

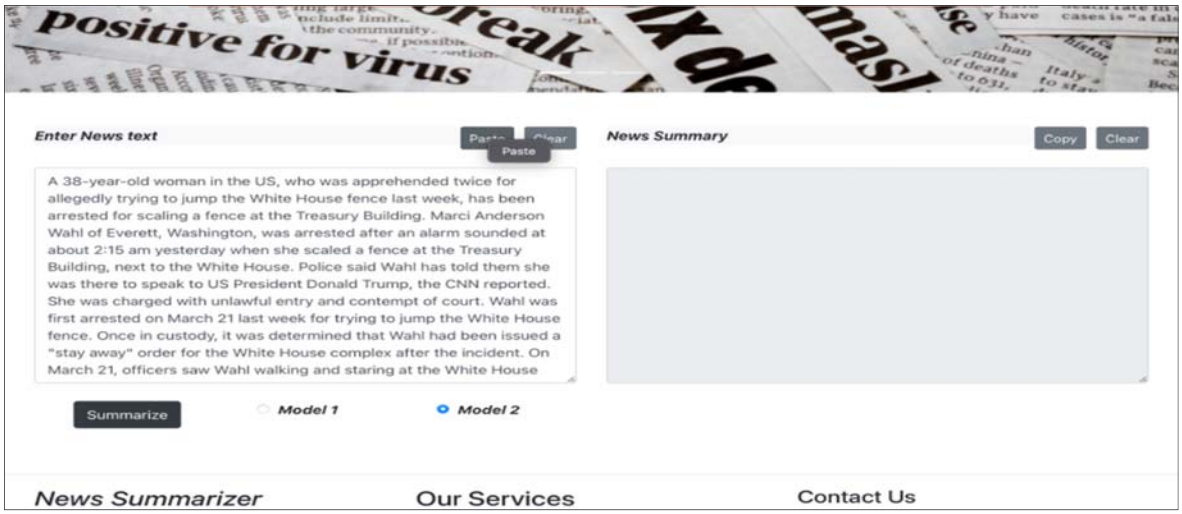


Figure 3: Now paste the desired text in the "Enter News Text" Box
The above figure explains the third step involved in generating the summary on the website.

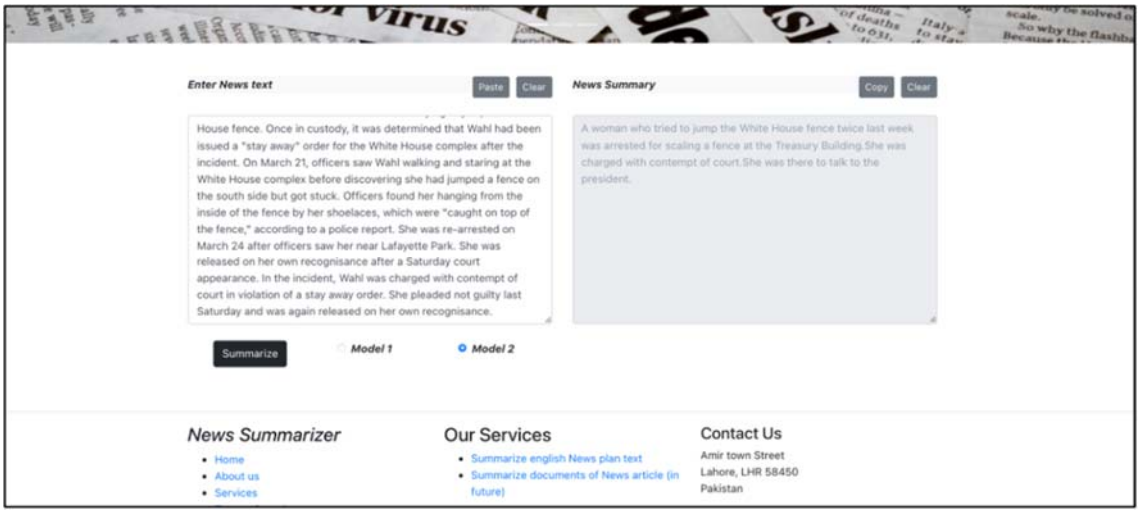


Figure 4: Click on the Summarize Button to generate Summary
The above figure explains the fourth and last step involved in generating the summary on the website

After these four steps shown in GUI, user shall be able to generate the summary of any news article by using any model available.

4.9 Database Design

In this section, we shall cover ER diagram which shows the main structure of our database design and relationship between different database tables.

4.9.1 ER Diagram

Following is the ER diagram for implemented website which shows that main database table, their attributes and their relation:

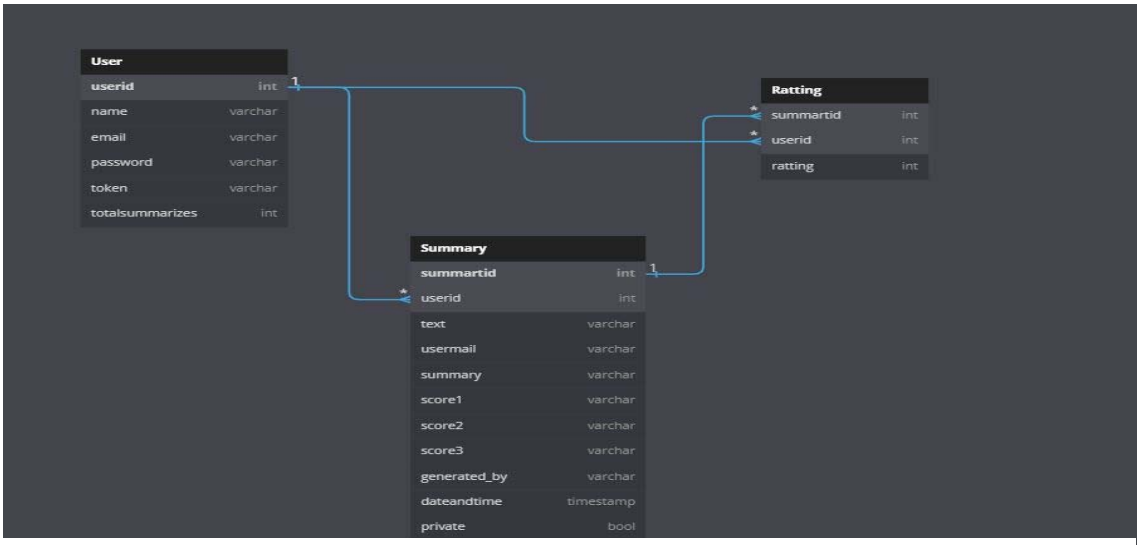


Figure 5: ER Diagram

4.10 Risk Analysis

Following is the risk analysis diagram which shows the risk, its severity, its impact and mitigation / prevention steps:

Table 2: Risk Analysis table

The risk analysis table which show the risks, severity, impact and prevention.

Risk	Severity	Impact	Mitigation/Prevention
Too many failed logins	High	A possible dictionary attack	Disable user account after a threshold level of failed attempts and use 2FA to restore it.
Sensitive data files being changed by privileged users.	High	User confidentiality compromised and loses trust in the platform.	Make sure permission changes are authorized.
Changes in schema/Design	Medium	Possible development cost overhead.	Make sure to write adaptable and scalable code.
Model(s) Generates less fluent summaries	Low	Reduction in market value	Human evaluation used as a finetuning variable.
File Server Crashes	High	Loss of important and user trusted data	Make regular backups.
Model generates inaccurate results	Medium	Credibility of the platform at stake.	Keep finetuning the model with new dataset given by user.
SQL injection	Low	Possible exposure to user information.	Use parameterized queries or prepared statements, along with strong validation

Chapter 5: Proposed Approach and Methodology

Since our project is largely exploratory, we have managed to touch various aspects of Abstractive text summarization using deep learning. In particular, we are using both transformer models and RNN based models. In particular, we used LSTM, GRU, SimpleT5, A Hybrid Transformer and BART model on CNN/Daily mail dataset to see how well each one performs. Furthermore, human evaluation has been conducted on two of our best models, SimpleT5 and Hybrid Transformer to compare how well each model holds. Following is a general pipeline of the basic building blocks applicable to all of our models:

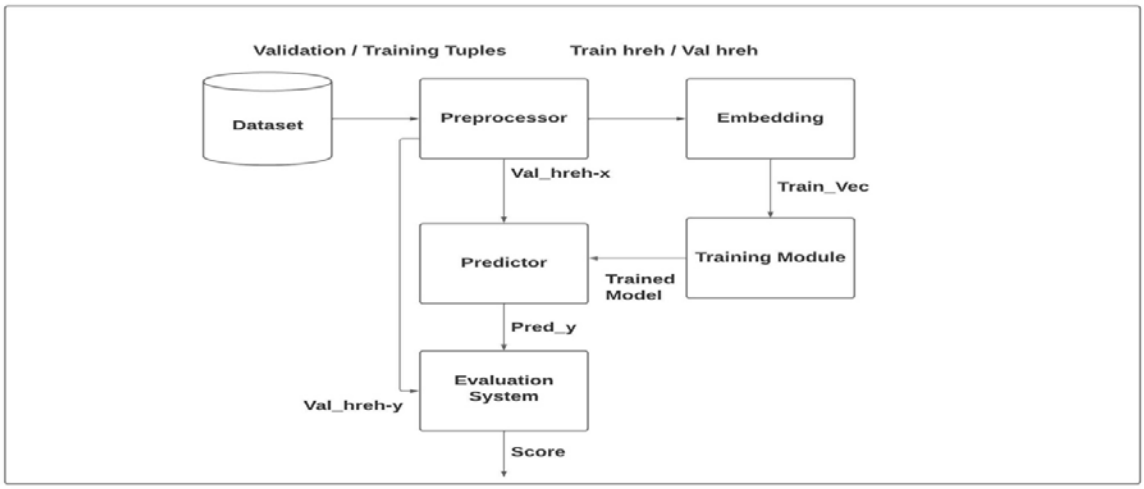


Figure 6: Research Methodology Diagram
The above figure explains all the steps involved in text summarisation.

Furthermore, we tried to explore a brief trade-off between abstractiveness of a generated summary and the factual inconsistencies it might produce during generation. This has been achieved using multitask learning, where the objective function of the model learns to do multiple tasks at the same time. In our case, we configured our model to generate both abstractive and extractive summaries at the same time. Furthermore, we tried to do the same thing using topic guided summarization.

We achieved this by creating a multipurpose dataset that contains the topic, abstractive summary and an extractive summary associated with the input text and doing multi task learning on the created dataset by one of the transformer models.

5.1 Dataset

The dataset we have used for our task is **CNN / Daily Mail**. The CNN / Daily Mail Dataset is an English-language dataset containing just over 300k unique news articles as written by journalists at CNN and the Daily Mail. The current version supports both extractive and abstractive summarization, though the original version was created for machine reading and comprehension and abstractive question answering. Furthermore, we have also created our own mini dataset consisting of 200 training examples belong to the category of computer science. Our dataset composes of both extractive and abstractive summaries, along with associated topics.

Furthermore, we have also created a very small dataset of our own with small technical paragraphs and their corresponding summaries in the technical domain. These input texts were extracted from computer science text books, each of these input texts, their topics and summaries were human written.

5.2 Authors of the Dataset:

The data was originally collected by Karl Moritz Hermann, Tomáš Kočiský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom of Google DeepMind. Tomáš Kočiský and Phil Blunsom are also affiliated with the University of Oxford. They released scripts to collect and process the data into the question answering format.

5.3 Statistics

CNN/Daily Mail:

- 300,000 training tuples
- Average Input text length: 512 tokens
- Average Summary Length: 120 tokens

Our Dataset:

- 200 training tuples
- Average input text length: 100 to 150 tokens
- Average summary length: 40 to 50 tokens

5.4 Preprocessing

The dataset we have collected needs to be preprocessed in order to be usable by deep learning models, before their vector representations can be generated.

Following diagram shows the steps involved in the pre-processing section:

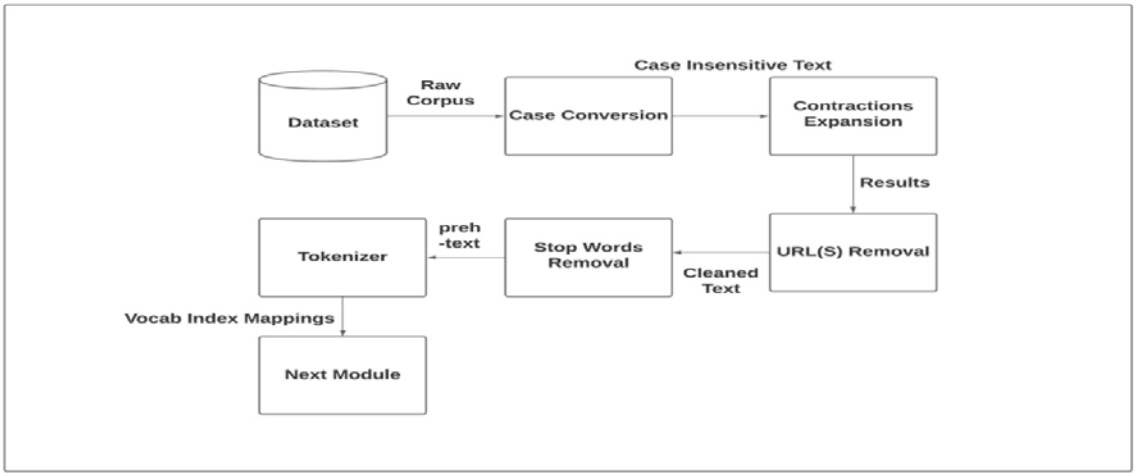


Figure 7: Preprocessing Method

The above figure shows a complete overview of pre-processing steps.

Following are the major preprocessing steps that are integral to our task:

1. **Case Conversion**: It is important to convert the entire body of text into a uniform representation by making it case insensitive and having any NLP(Natural language processing) model not distinguish same words with different cases. In practice, usually text is converted into lower case.
2. **Contractions Expansion**: Any language model needs the language it wants to understand to be in its simplest possible form. Such ambiguities that are not otherwise understood by a model can hinder the model in its effective learning and learn unneeded contexts. For example: Could've might be considered a singular word when it's a combination of could and have and might be distinguished from could in its singular form.
3. **Removing URLs:** Since we are dealing with news dataset, referring to other sources in most online articles shouldn't be left untreated. Hence, to enable our language model to not consider URLs as part of the language its trying to learn, we need to remove such unwanted strings of characters such as URLs.
4. **Stop Words Removal:** Lastly stop words are also removed from the text that have no actual semantic relevance to the crux of the input text. Such as 'a', 'about', 'am' and 'the'.
5. **Tokenization:** No deep learning model understands text in the form they are given. They need to be converted into a fixed vocabulary set of size K and then changed to numbers. The process is fairly intuitive: create a vocabulary from the corpus and assign a unique index(number) to each word. The reason why these numbers are important are made clear in the word embeddings section.

5.5 Word Embeddings

Before our neural network model can begin learning, it needs to be fed a vector representation of corpus words into the input layer. There are a number of notable techniques that are used for this purpose but we have chosen the following two:

5.5.1 Word2Vec

Word2Vec is an unsupervised learning shallow neural network. What it tries to do is learn a vector space by considering surrounding input words in relation to a word at position p within a fixed window size k. The resulting vector is the hidden layer extracted after the model is fully trained. The input words are first converted into their one hot encoded representations. The network has two variations to achieve this: Skip gram and CBOW.

1. **Skip Gram:** Skip gram takes as input a word at position p and predicts the words in its surrounding window.
2. **CBOW:** CBOW on the other hand takes the surrounding words in this window as input and predicts the word at position p.

The difference is made even more clear with the help of the following diagram:

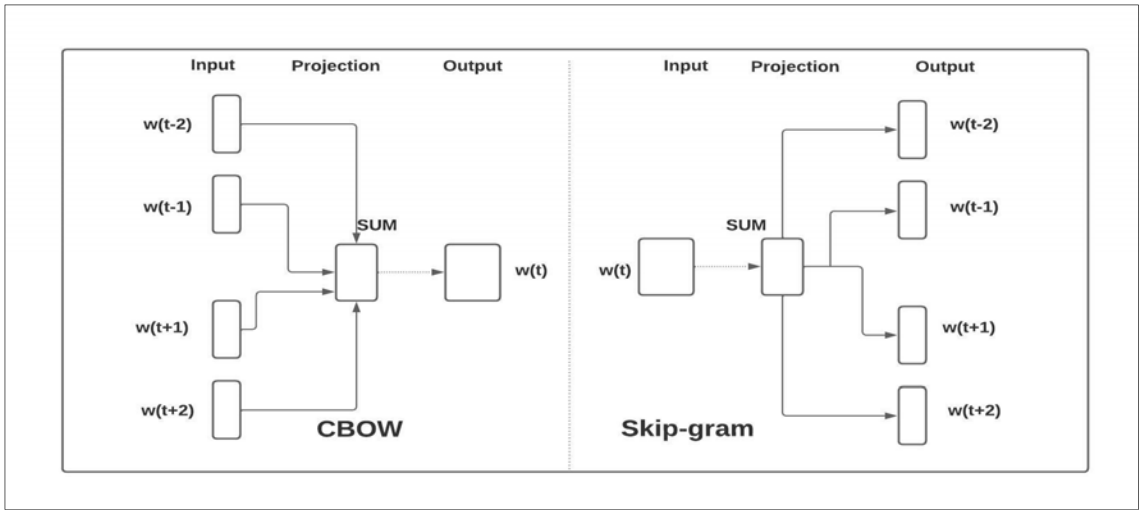


Figure 8: Word2Vec Architecture
This above figure shows the architecture of Word2Vec embeddings.

5.5.2 Glove

Glove too uses a neural network to learn such contextual embeddings but considers co-occurrence of any two words in an entire corpus. The co-occurrence between each word pair in the corpus is simply the frequency of such pairs appearing.

For this too, a fixed window size is defined and a matrix is created as follows:

Table 3: Matrix for Glove Model
Matrix table where Wi is the ith word in the vocabulary and C(i,j) is the cooccurrence finding words i,j in the corpus

	W1	W2	W3	W4
W1	C(1,1)			
W2	C(2,1)	C(2,2)		
W3	C(3,1)	C(3,2)	C(3,3)	
W4	C(4,1)	C(4,2)	C(4,3)	C(4,4)

These co-occurrences are simply learnt using a neural network and the resulting hidden layer is extracted as the vectorized transformation of the word.

5.6 Training Models

We have chosen two neural language modelling architectures to generate summaries upon which we will experiment and observe their resultant accuracies:

5.6.1 Encoder Decoder Recurrent neural networks

Encoder decoder takes two pairs of RNNs units: one for encoding the input text in the form of a fixed length context vector and another sequence of RNNs for taking this context vector as input and generating a summary. The former and latter are known as encoder and decoders respectively. However, the architecture is prone to many issues involving long sequence temporal dependencies which are resolved via many enhancements to solve its inherent

problems. For example, for a long sequence input text, a fixed length context vector might contain insufficient information to be fed to decoder. And secondly, the sequential nature of the network makes it impossible for parallel optimization. At first we tried LSTM model using 2 word embeddings. We then try both GRU and LSTM based model with incorporation of attention mechanism.

The architecture of the model is made even more clearer using the following diagram:

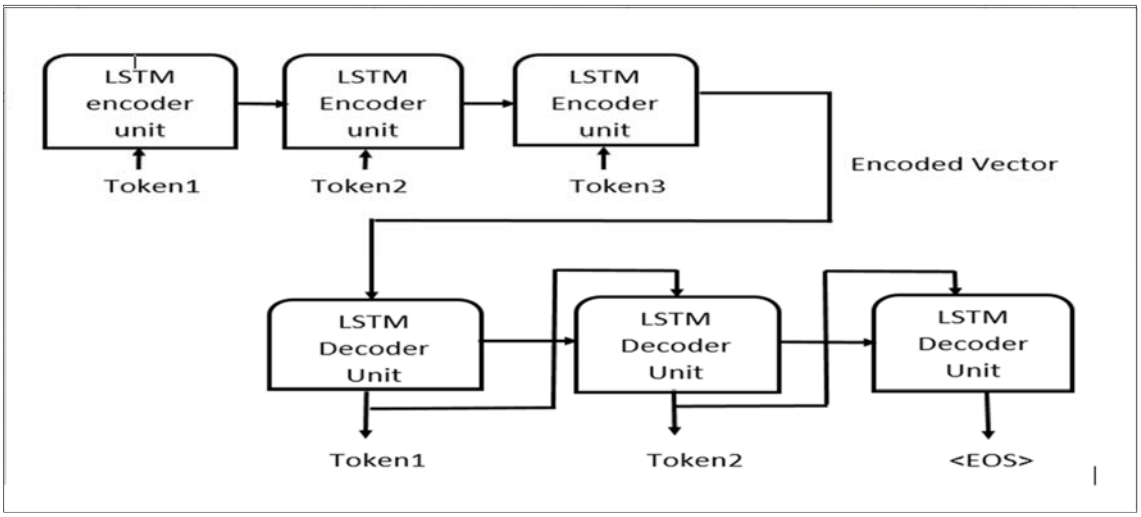


Figure 9: Encoder-Decoder RNN
This above figure shows the working of Encoder-Decoder RNN.

5.6.2 Transformer model

Transformer architecture on the other hand takes this entire sequence in a one-shot manner rather than processing it in a tokenized manner. The architecture too uses an encoder and decoder but without employing the recurrent structure of RNN based neural encoder decoders. This makes the architecture very suitable for parallel optimization and also makes it easy to map long sequences into a rich context vector. This is achieved via self-attention mechanism, where one encoding layer attends to previous encoding layer, enabling it to understand the temporal dependencies without necessarily employing a recurrent or time dependent architecture like RNNs.

Following are the transformer model based training methodologies we tried:

1. **SimpleT5:** The models work by encoding the input text via cross attention layers and then feed it to decoders. The decoder then generates auto-regressive output. The output of the decoder block is than given to the final layer. This final layer is also known as dense layer and uses SoftMax activation function for generating output. Finally, the weights of the output of final layer is again fed into the embedding matrix in order to produce the final summary.
2. **Hybrid T5 + Pegasus:** Two models in conjunction were used during prediction: namely, T5 extractive summarizer and Pegasus paraphrasing transformer. However for training, the T5 model and Pegasus were trained separately.
3. **BART:** This is another generative transformer based model which we used for generating abstractive summaries, which follows similar training steps to T5 model.

4. **Multitask learning:** We used the above SimpleT5 model finetuned using multitask learning by providing input signals in the input text to guide the model to generate both generate two kinds of sequences. Following is a block diagram of the system we developed:

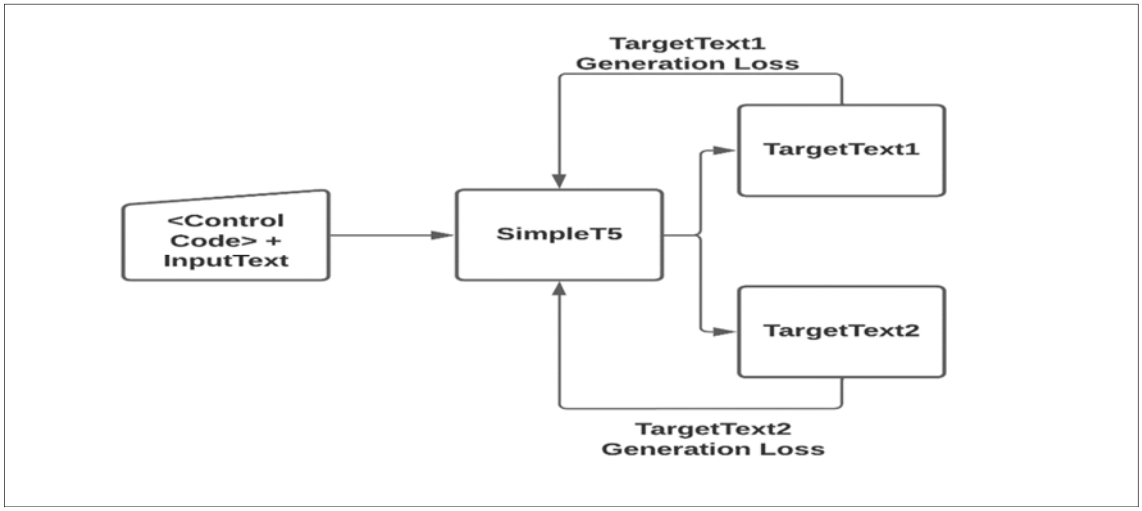


Figure 10: Multitasking System
The above figure shows the architecture of multitasking system that we developed.

The following high level diagram describes the overall architecture:

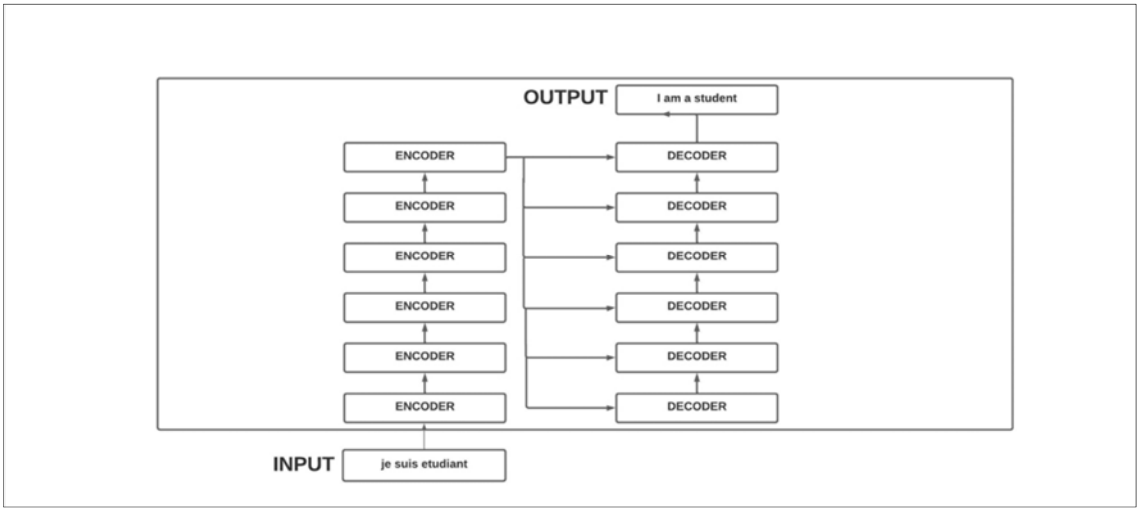


Figure 11: Architecture of Transformer Model
The above figure shows the architecture of transformer model.

5.7 Prediction

5.7.1 RNN based models

After the model has been trained, all that needs to be done is feed the model with an input text so it can generate a summary. In decoder every unit is dependent upon the output of the previous one to predict the next word in the sentence, which is simply a classifier at each unit predicting the next word out of the vocabulary set in the SoftMax layer. The output is generated

one token after another in a fixed number of timesteps which is the maximum output length we have already defined before training, or till an EOS(end of sequence) token is encountered.

5.7.2 Transformer model

The prediction mechanism with transformers is slightly trickier. Even though during training, the encoder takes the input representation in a one shot manner, the decoder however, still does it in a sequential manner to ensure that all the temporal dependencies are validly reproduced during prediction. It takes the last generation matrix and encoding matrix as input to get the next predicted matrix of words containing the output sequence. During prediction, it simply needs to take the encoding matrix.

5.7.3 Multitask Learning

For multitasked learning, we simply had to prepend the control code(s) we used during training to indicate the kind of sequence we want to generate during prediction. For example, prepending <Topical> to input text will generate a topic or prepending <Abstractive> will generate an abstractive summary.

5.8 Evaluation

5.8.1 Automatic Evaluation

We perform evaluation of our summaries using the rogue evaluation metric which is the standard evaluation metric used in all previous studies for abstractive text summarization.

It is essentially a set of metrics for evaluating automatic summarization of texts as well as machine translations. What it does is compute a ratio of overlapping words Wo between predicted and reference summary to total words in the reference summary Wr. A value between 0 and 1 is generated, where a value of 1 indicates close to exact match and a value of 0 makes it completely different. The formula to compute this score is as follows: **Rogue = Wo/Wr**

Following is the block diagram for reference:

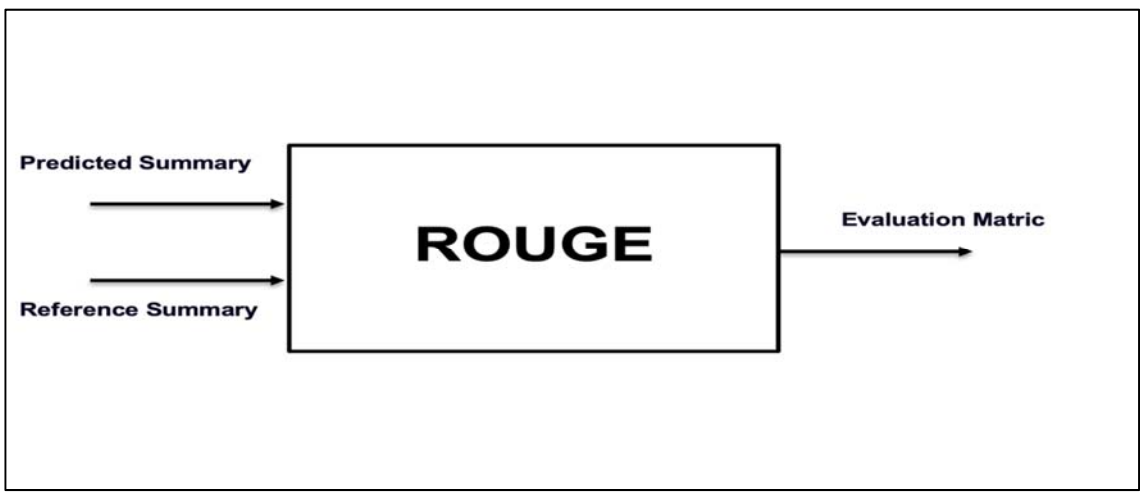


Figure 12: Block Diagram about ROUGE Score
The above figure shows the working of ROUGE in evaluating summary.

5.8.2 Human Evaluation

Automated evaluation has its limits and with abstractive summarization which is prone to be influenced by human biases such as how well written and easy to understand a text looks or fluidity of language and expression, we also conducted a brief survey to perform human testing on our models. Both the input text and output summaries are provided for each model and we ask them to rate it on a linear scale. We considered both the educational backgrounds and literacy of our audience to get a fair evaluation on our summaries. The scale defined for rating our summaries was a linear scale from 1 to 5 defined as follows:

1. Gibberish
2. Readable Only
3. Readable and covers some information
4. Covers important information but not very readable
5. Perfect

Chapter 6: High-Level and Low-Level Design

The high and low-level design of the system is given below:

6.1 System Overview

The software system developed for this project is a web-based platform that generates abstractive summaries of user-provided text using deep learning techniques. The platform is designed to provide users with a simple and intuitive interface for generating summaries of news articles, research papers, and other types of text documents.

6.2 Design Considerations

Before developing the software system for abstractive text summarization, several key design considerations were taken into account. These include issues related to the accuracy and efficiency of the summarization process, as well as matters related to user interface design and system scalability.

One of the primary considerations in the design of the system was the accuracy and efficiency of the summarization process. To ensure that the generated summaries are of high quality, the system uses state of the art deep learning models which have been shown to be effective in generating high-quality abstractive summaries. The models are trained on a large dataset of CNN/Daily Mail news articles, which ensures that it is capable of generating accurate summaries of a wide range of text documents.

In addition to accuracy, the system was also designed with efficiency in mind. The deep learning model is optimized for speed and can generate summaries in real-time, allowing users to quickly and easily summarize text documents without having to wait for long processing times.

6.2.1 Assumptions and Dependencies

Before designing and developing the software system for abstractive text summarization, several key assumptions and dependencies were taken into account. These include issues related to related software or hardware, operating systems, end-user characteristics, and possible and/or probable changes in functionality.

The software system for abstractive text summarization assumes that users have access to a modern web browser, such as Google Chrome or Mozilla Firefox, to access the website. Additionally, users should have a stable internet connection to ensure that the website loads quickly and efficiently.

The software system is designed to be platform-independent and is compatible with a wide range of operating systems, including Windows, macOS, and Linux. The system assumes that users have a basic understanding of how to use a web browser and interact with a website. The user interface is designed to be intuitive and user-friendly, with clear and concise labeling and intuitive navigation controls. However, the system does not provide extensive user training, and users are expected to be able to operate the system with minimal guidance.

6.2.2 General Constraints

The software system for abstractive text summarization is subject to several global limitations and constraints, which have a significant impact on its design and implementation. These

constraints are imposed by various factors, including hardware and software environment, end-user environment, availability and volatility of resources, standards compliance, and security requirements.

The software system for abstractive text summarization requires a modern computer or device with sufficient processing power to ensure optimal performance. Specifically, the system requires a processor with at least 2 cores and 4 GB of RAM. Additionally, the system requires access to a modern web browser, such as Google Chrome or Mozilla Firefox, to access the website.

The system is designed to be highly performant, with a focus on delivering fast and accurate abstractive text summarization results. However, due to the complexity of the underlying deep learning models used in the system, there may be performance limitations when processing very large documents or when serving many users concurrently. To mitigate these performance limitations, the system uses caching and parallel processing techniques to distribute processing load across multiple CPU cores and servers.

The system is designed to be secure and reliable, with a focus on protecting user data and ensuring the integrity of the system. To achieve these goals, the system uses a combination of encryption, secure authentication, and access control techniques. Additionally, the system is designed to be highly resilient in the face of potential security threats, with extensive logging and auditing capabilities to detect and respond to security incidents.

6.2.3 Goals and Guidelines

In the case of the abstractive text summarization system using deep learning tools, the following goals and guidelines are relevant:

1. **Emphasis on accuracy and coherence of summaries:** The primary goal of the system is to generate accurate and coherent summaries of input text. This is important to ensure that users can rely on the system to provide them with useful and reliable summaries.
2. **Focus on natural language generation:** The system should generate summaries that are grammatically correct, well-formed, and use natural language that is easy to understand for the end-users.
3. **Efficiency:** The system should be designed to operate efficiently and generate summaries in a timely manner. This is important to ensure that users do not have to wait long for their summaries to be generated.
4. **Scalability:** The system should be scalable to handle large amounts of input data and generate summaries quickly and accurately.
5. **Ease of use:** The system should be designed to be user-friendly, with an intuitive interface that makes it easy for users to input text and receive summaries.
6. **Adherence to best practices:** The system should be designed in adherence to best practices in software engineering, including good coding practices, proper version control, and reproducibility of results.

6.2.4 Development Methods

The development method used for the design of the abstractive text summarization system using deep learning tools is an iterative and incremental approach. This approach involves the following steps:

1. **Requirements gathering:** The system requirements were identified through discussions with the advisor, a review of the literature, and an analysis of existing systems.
2. **Design:** The system architecture and design were developed based on the requirements identified in the previous step. This involved selecting appropriate deep learning frameworks and libraries, and developing the user interface.
3. **Implementation:** The system was implemented using Python programming language, with the deep learning libraries Pytorch and Transformers library. The user interface was developed using React framework.
4. **Testing:** The system was tested using various datasets such as the CNN/Daily mail dataset to evaluate its accuracy, coherence, and efficiency. A range of test cases were developed to cover different input text scenarios and edge cases.
5. **Deployment:** The system was deployed to a local server, where it can be accessed by users through a web browser.

6.3 System Architecture

The system architecture for the abstractive text summarization website will be designed using a microservices architecture. The microservices architecture is a design pattern that decomposes the system into loosely-coupled and independently deployable services. Each service will be responsible for a specific functionality of the system.

At the top-most level, the system will consist of the following components/subsystems:

1. **User Interface:** This component will be responsible for managing the user interactions and providing a web-based interface for users to input the text to be summarized.
2. **Text Summarization Service:** This component will be responsible for the core functionality of the system, which is generating abstractive summaries of user-provided text. It will utilize different deep learning models to generate the summaries.
3. **Database:** This component will be responsible for storing the user data and generated summaries.
4. **Authentication and Authorization Service:** This component will be responsible for managing user authentication and authorization to access the system.

The components will communicate with each other using REST APIs. The User Interface will communicate with the Text Summarization Service to request summaries for user input. The Text Summarization Service will communicate with the Database to store the generated summaries. The Authentication and Authorization Service will handle user authentication and authorization requests from the User Interface.

The use of microservices architecture has several benefits, including scalability, flexibility, and maintainability. It allows for the independent development, deployment, and scaling of each component, which helps to optimize the system's performance and reliability.

6.4 Architectural Strategies

The following strategies were used in designing the architecture of the software system:

6.4.1 Use of Object-Oriented Programming (OOP) paradigm

The software system is designed using the OOP paradigm, which emphasizes the use of objects to represent data and functionality. This allows for better encapsulation and modularity of the system, making it easier to maintain and extend.

6.4.2 Modular design

The system is divided into modules, each responsible for a specific functionality. This allows for easier maintenance and testing of the system.

6.4.3 Scalability

The system is designed with scalability in mind, allowing for easy expansion and adaptation to changing requirements.

6.5 Domain Model/Class Diagram

Following is the class diagram for the system:

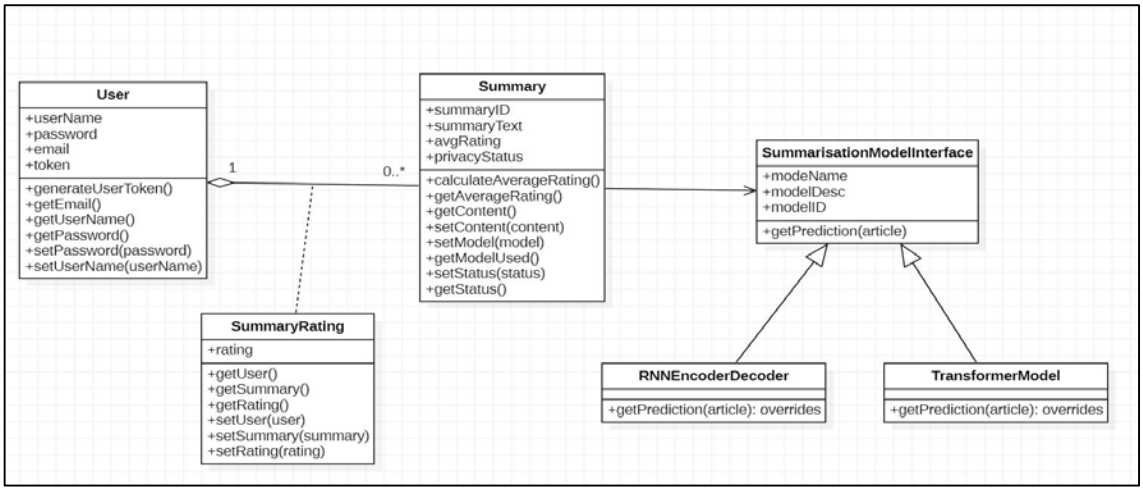


Figure 13: Class Diagram of the system

The above figure shows the class diagram of abstractive text summarization using deep learning.

6.6 Sequence Diagrams

Following are the sequence diagrams for the given system:

These diagrams shows different sequence diagrams for cases such as login, signup, generating summary, etc. They are given below:

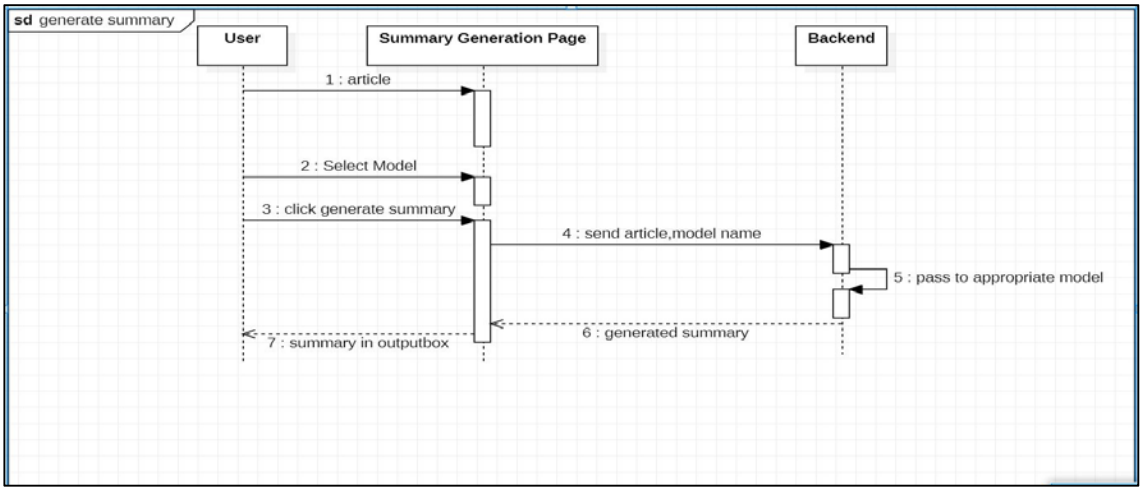


Figure 14: Sequence Diagram for Generate Summary
The above figure shows the sequence diagram for generate summary .

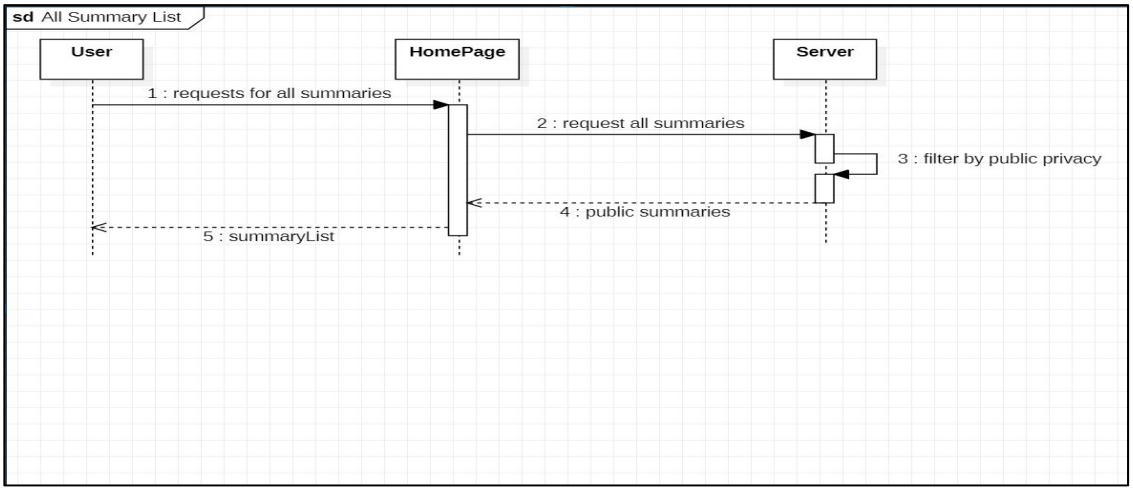


Figure 15: Sequence Diagram for All Summary List
The above figure shows the sequence diagram for all summary list .

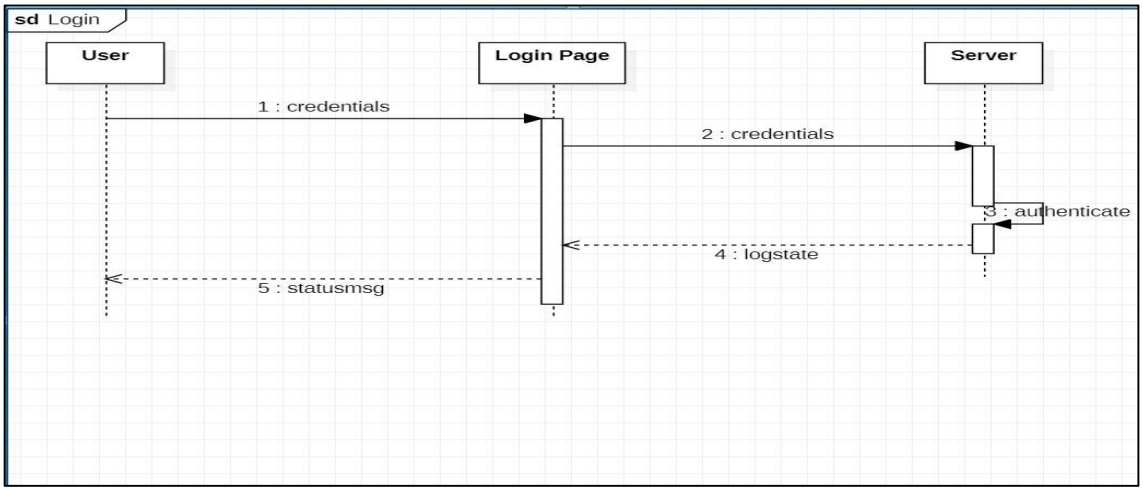


Figure 16: Sequence Diagram for Login
The above figure shows the sequence diagram for login .

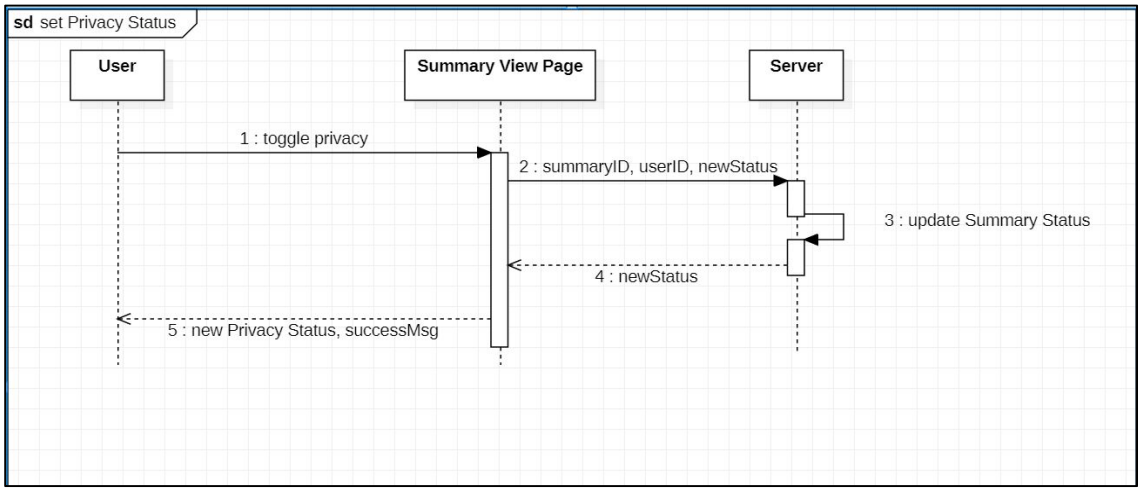


Figure 17: Sequence Diagram for set privacy status
The above figure shows the sequence diagram for set privacy status.

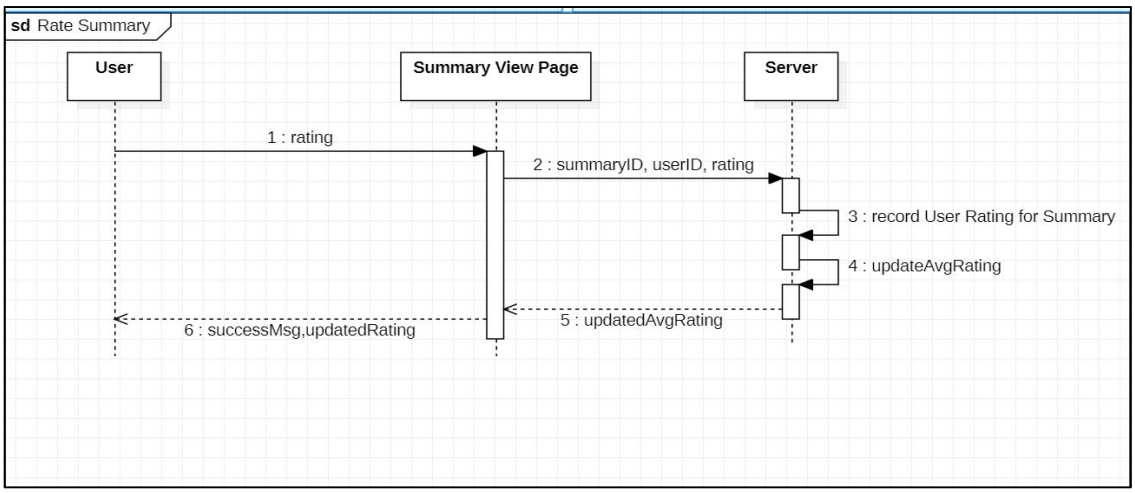


Figure 18: Sequence Diagram for Rate Summary
The above figure shows the sequence diagram for rating summary.

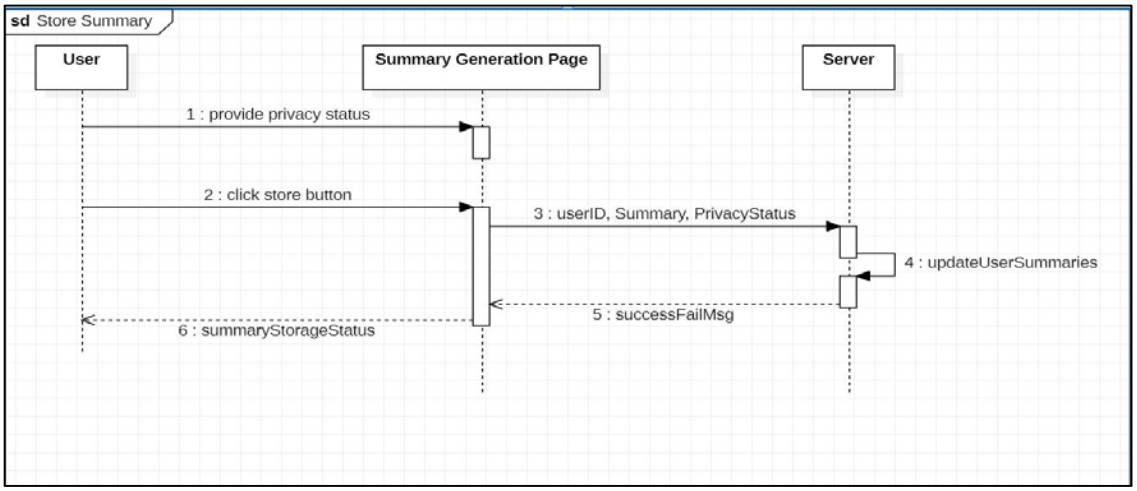


Figure 19: Sequence Diagram for Store Summary
The above figure shows the sequence diagram for storing summary.

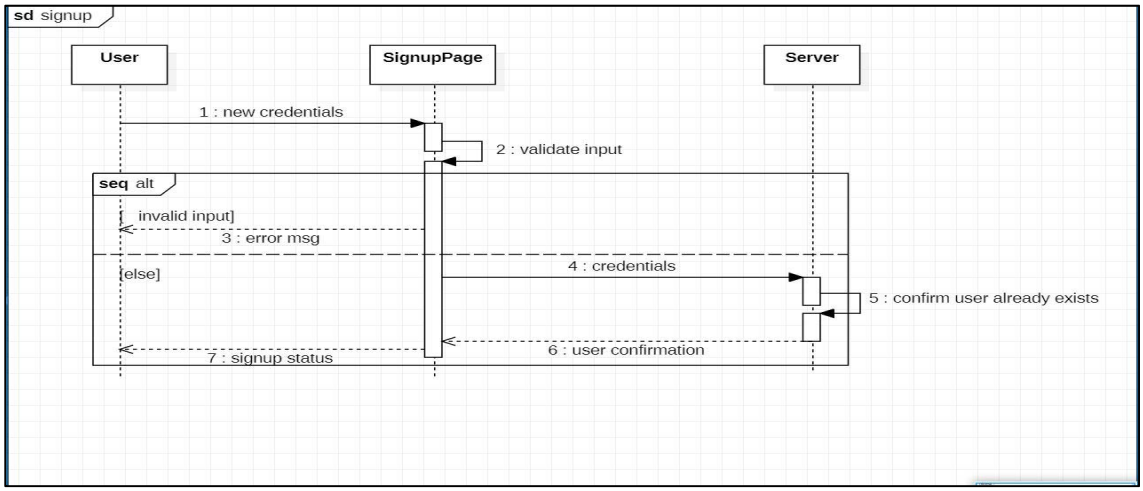


Figure 20: Sequence Diagram for Signup
The above figure shows the sequence diagram for signup.

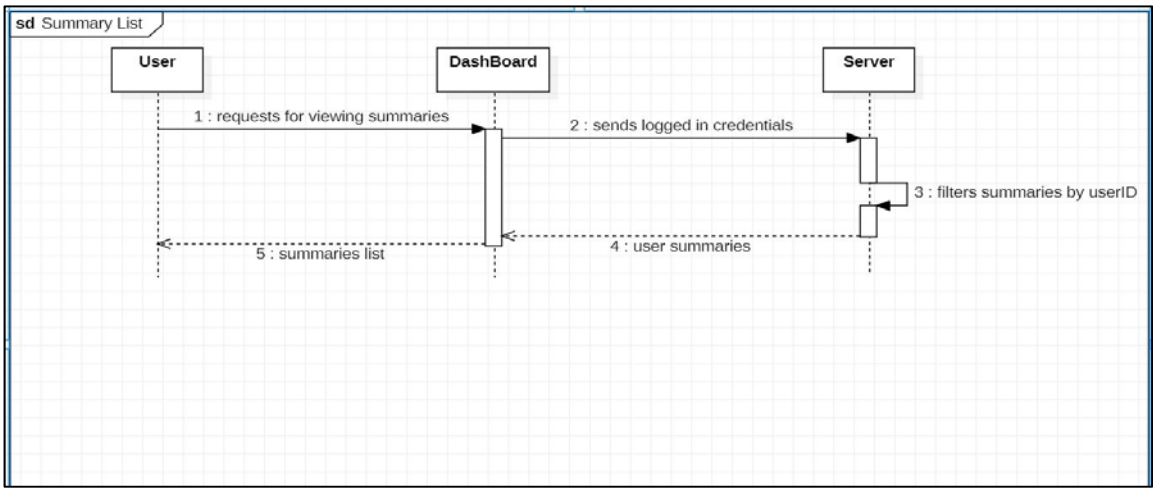


Figure 21: Sequence Diagram for Summary List
The above figure shows the sequence diagram for summary list .

6.7 Policies and Tactics

Following are different policies and tactics that can be considered:

6.7.1 Coding guidelines and Conventions

One important policy that can improve the quality and maintainability of software is the establishment of coding guidelines and conventions. These guidelines can cover naming conventions, commenting practices, variable and function naming conventions, and code formatting rules. Such guidelines can help ensure that the code is consistent and readable, which makes it easier for developers to work on the codebase, and reduces the likelihood of bugs being introduced. Adhering to coding standards also facilitates better collaboration and code reviews between developers.

6.7.2 Testing Strategy

A testing strategy defines the approach that the development team will take to ensure that the software is tested thoroughly, and that it meets the specified requirements. The testing strategy should consider the types of tests that will be performed, such as unit, integration, and system tests, as well as the tools and techniques that will be used. The testing strategy should also include a plan for how testing will be conducted throughout the development cycle, including who will be responsible for testing and when the testing will occur. A well-defined testing strategy can help ensure that the software is of high quality and that any defects are caught early in the development cycle.

6.7.3 Version Control Policies

Version control is a crucial tool for managing changes to the software over time. Policies and tactics for version control should be established to ensure that the team can work effectively with the version control system. These policies and tactics can include branching and merging strategies, naming conventions for branches and tags, policies for commit messages, and procedures for code review and release management. By establishing clear policies and tactics for version control, the development team can ensure that changes to the software are managed effectively, and that the team can work collaboratively without introducing conflicts or breaking the build.

Overall, these policies and tactics are critical for ensuring that the development team can work effectively, efficiently, and collaboratively to produce high-quality software that meets the specified requirements.

Chapter 7: Implementation and Test Cases

We developed our deep learning models using python libraries on google Colab platform and trained them on p800 GPUs. Specifically, we have developed encoder decoder and transformer models, using TensorFlow and Pytorch. An experiment was also conducted by developing a language translation algorithm that maps our summarization task onto the translation task. This tool too was based on encoder decoder model and uses TensorFlow for its underlying backend logic.

7.1 Implementation

7.1.1 Encoder-Decoder model

Following are the encoder-decoder models that we have implemented for abstractive text summarization:

Vanilla Encoder-Decoder Model

In this model, input layer is simply vectorized representation of the input text and the embedding layer takes the input word and generates a vector to be fed to encoder. The encoder encodes the sentence received and passes it to a decoder. The decoder also receives an embedding layer for the target sequence.

Following are the components of encoder decoder deep learning model we have implemented, illustrated via the following block diagram:

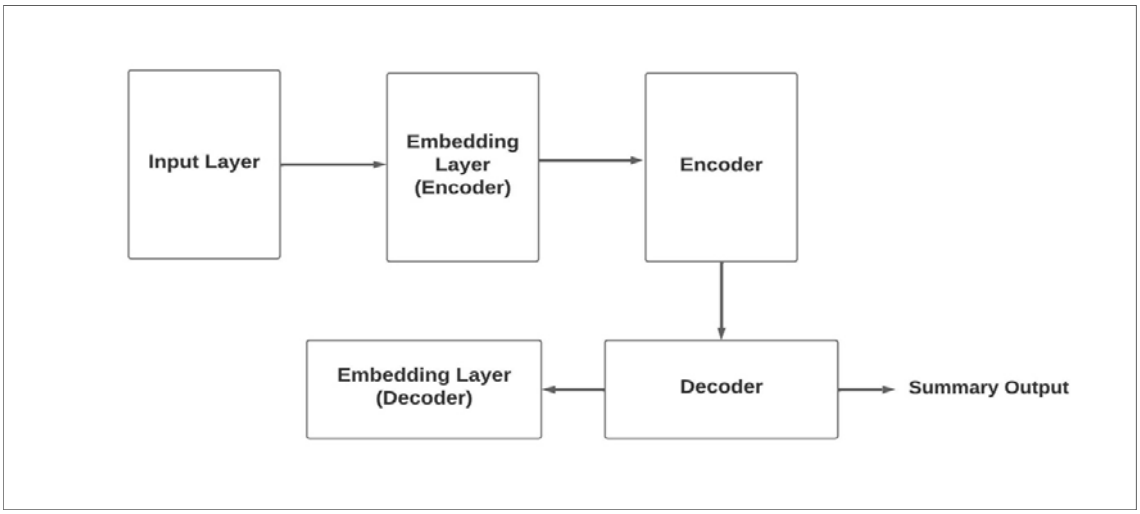


Figure 22: Diagram of Vanilla Encoder-Decoder Model
The above figure shows the working of Vanilla Encoder-Decoder model for abstractive text summarization

We made following experimentations with changes to the above embedding layer of the model:

Glove embeddings

We used pretrained glove embeddings provided by Stanford and extracted these word vectors from the following publicly available text file on Kaggle: “glove.6B.100d.txt”. The file contains 6 billion example tokens with their corresponding 100 dimensional vector representations as follows, having the following file structure:

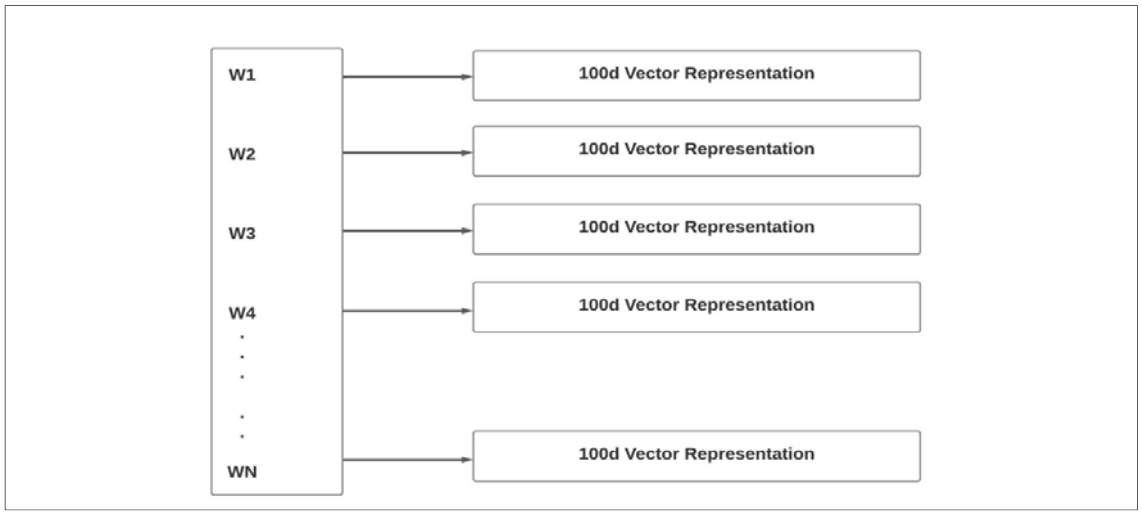
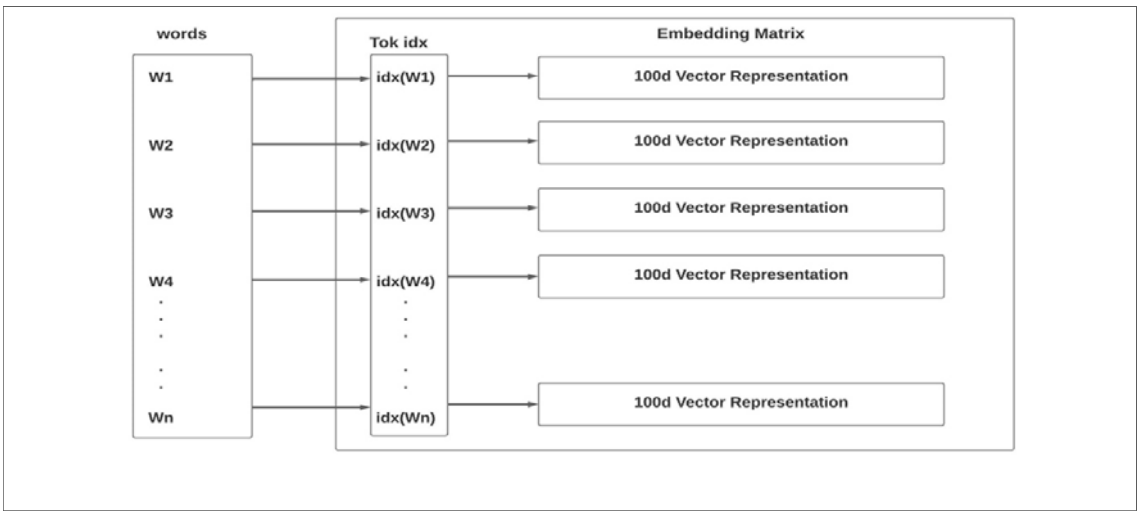


Figure 23: Diagram of File Structure of Glove Embedding
The above figure shows the file structure of Glove Embeddings where each vector represents 100 dimensions

This text file was parsed and converted into a dictionary of word vectors, only filtering out the words relevant to our corpus to create a resultant token vector matrix. Each word was mapped to an integer token which became the index of the resultant glove embedding matrix using Keras tokenizer class. This matrix is input to the embedding layer and the layer is frozen such that the embedding layer doesn’t learn the embeddings on its own and uses our embeddings instead. The process can be described via the diagram shown above.



The above figure shows the working of embedding layer of Glove model and the resultant Glove embedding matrix

Word2Vec

Similar to glove embedding, we create another embedding matrix using word2vec algorithm provided by genism library. The additional step for this embedding is the training since pretrained embeddings unlike glove aren't available so we have to create them from our corpus. The algorithm trains on our corpus and generates the vector representations for each word similar to the glove 6B text file. Following is the diagram showing word2vec algorithm:

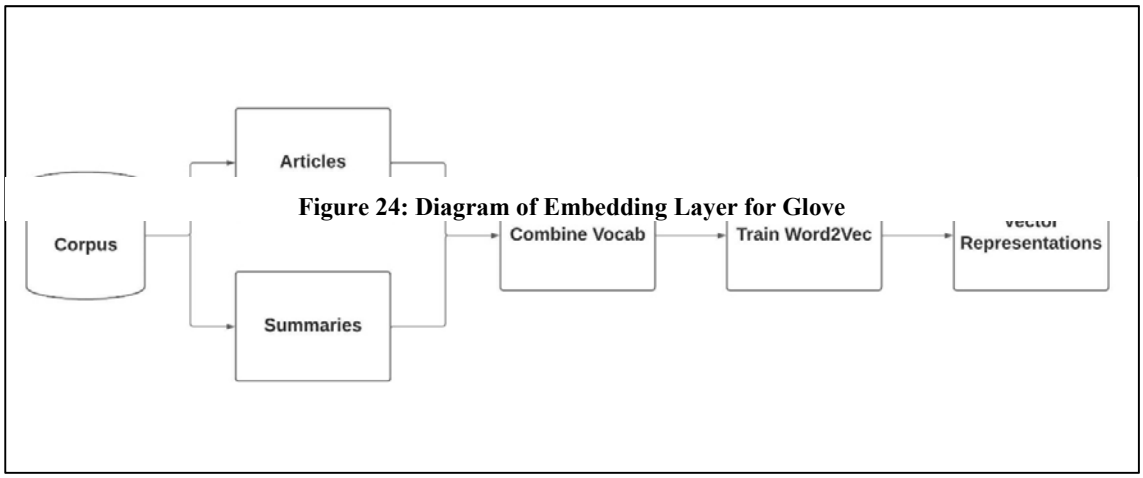


Figure 24: Diagram of Embedding Layer for Glove

Figure 25: Diagram of Word2Vec Model

The above figure shows the working of Word2Vec model from corpus generation till the generation of vector representations.

The rest of the procedure for constructing the matrix and feeding it to the embedding layer is same as that of glove.

Language Translation Model

We also tried mapping the task of translation to the task of summarization by treating summary as the target language we want to generate and using articles as the context language fed to the translator. For this we also used an encoder decoder with an addition of attention layer to try and improve the coherence of our summary output sentences. And for performance consideration, we employed teacher forcing mechanism allowing the translator to use parallelization during training time in contrast with the strictly sequential structure of the vanilla encoder decoder.

Following is the diagram showing the basic structure of language translation model:

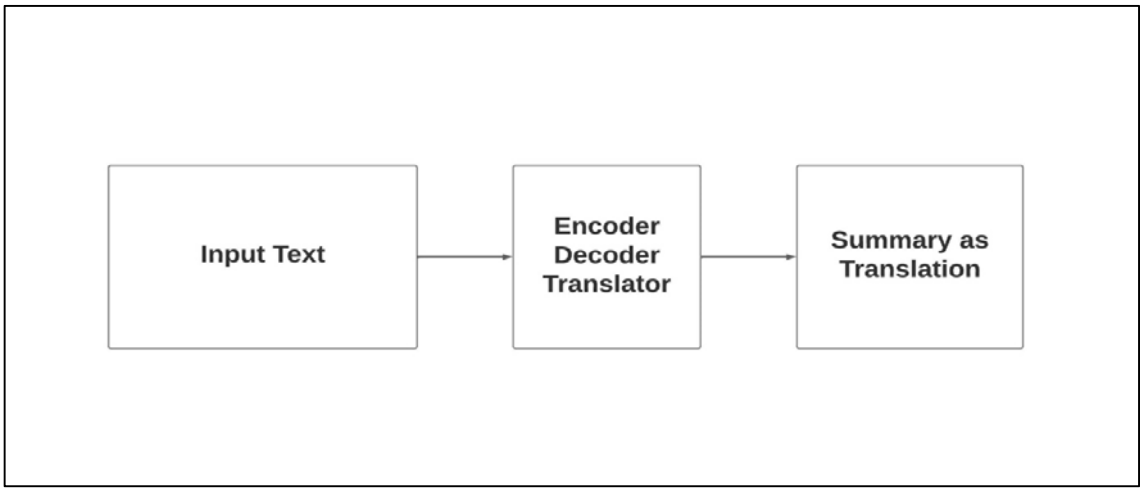


Figure 26: Diagram of Language Translation Model
The above figure shows the basic structure of language translation model which has three main modules.

In the diagram shown below, the matrix above is the attention matrix attached to the decoder which attends to the outputs of the encoder in order to ensure a greater deal of contextual richness.

This model too was implemented using TensorFlow with components similar to above approaches with only two changes:

1. Addition of attention layer as described above
2. Unfreezing the embedding layer and letting it learn the embedding vectors on its own.

Following is the diagram showing our adapted summarization model:

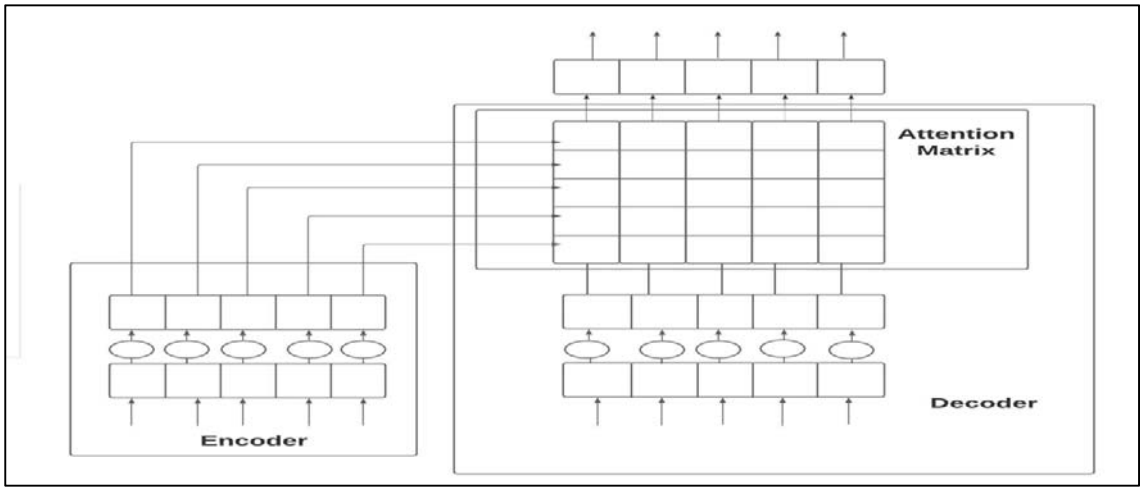


Figure 27: Diagram of Adapted Summarization Model
The above figure shows the working of our adapted summarization model which was implemented using TensorFlow

Since both encoder decoder will be treating the articles and their summaries as separate languages, therefore, by providing them with our own fixed embeddings, we can confuse the translator into thinking of the two as the same language.

7.1.2 Transformers Model

Transformer model is the modern representation of encoder-decoder model as it has more powerful and high performance mechanism for text summarization as compared to other models. This model was proposed in 2017 in the paper "Attention is all you need". In 2018, first transformer based model was proposed which is now commonly known as BERT. For the implementation phase, we have chosen T5 transformer model as it shows the best Rouge-1 and Rouge-L score out of all other transformer models being discussed in the literature review section [6].

Simple T5 Transformer Model

T5 model was developed by Google AI team in 2020 for the purpose of natural language processing. The models work by encoding the input text via cross attention layers and then feed it to decoders. The decoder then generates auto-regressive output. The output of the decoder block is than given to the final layer. This final layer is also known as dense layer and uses SoftMax activation function for generating output. Finally, the weights of the output of final layer is again fed into the embedding matrix in order to produce the final summary. The figure below shows the working of T5 transformer model:

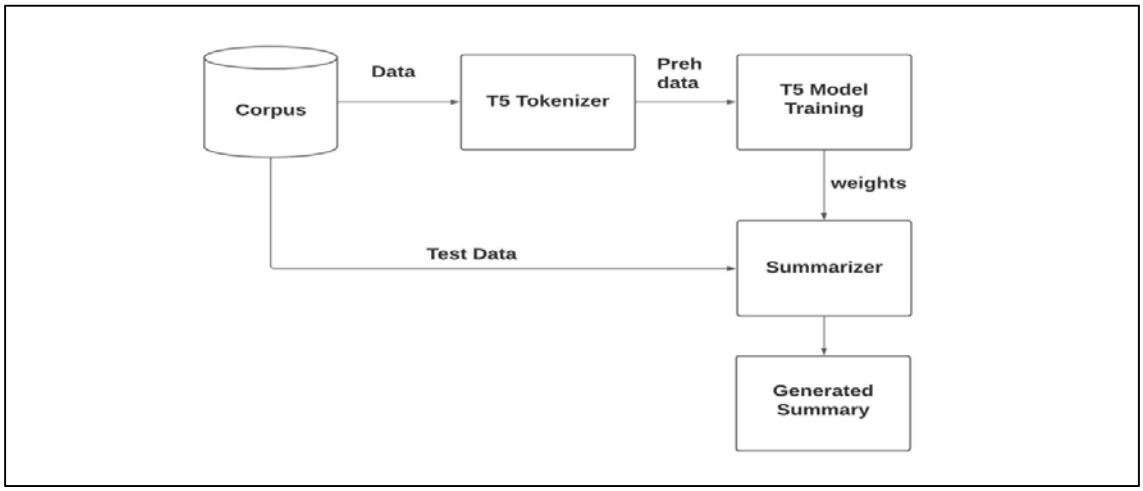


Figure 28: Diagram of T5 Transformer Architecture
The above figure shows the working of T5 transformer model and different modules involved in it.

For implementing Simple T5 model, following steps were performed:

Following steps describe the basic structure of the working code for T5 transformer model:

1. Firstly, all important libraries such as SimpleT5, Rouge and Skit learn were installed. These libraries were downloaded using pip command in terminal.
2. Then, all important functions and classes from the above mentioned libraries were imported. These functions will help in training the model.
3. Next, the dataset file named as train.csv from CNN/Daily Mail dataset was imported and all relevant text and summaries were extracted from it respectively.

4. Next, the training function was made using Simple T5 function and epochs size was set to 3. The batch size was set to 16.
5. Finally, the model was trained using the above function and summary was generated.

Hybrid T5 Transformer Model

Pytorch lightning: This library is used by the researches to make the python machine learning model more efficient and faster. It provides different functions for training the model effectively. It is mostly used for deep learning projects such as text summarization, objects detections, generating automated captions, etc. For the implementation of T5 model, we have imported data loader, model checkpoint, and trainer function from this library.

*Transformers Library***:** This transformer library consists of "over 30 pre-trained models and 100 languages, along with eight major architectures of natural language processing (NLP) and natural language generation (NLG). " For the implementation of T5 model, latest version of transformer library which is "4.20.1" was used. From the library, we have imported important classes such as T5TokenizerFast, T5ForConditionalGeneration and AdamW.

1. Firstly, all important libraries such as Transformers, Rouge and Pytorch Lighting were installed. These libraries were downloaded using pip command in terminal.
2. Then, all important functions and classes from the above mentioned libraries were imported. These functions will help in training the model.
3. Next, the dataset file named as news-summay.csv was imported and all relevant text and summaries were extracted from it respectively.
4. In the next step, we loaded the pretrained tokenizer named as T5Tokenizer.
5. Now, all relevant classes such as "NewsSummaryModel" was made using Pytorch Lightning. These classes were used to generate the final summary with proper syntax.
6. Next, the training function was made using Pytorch Lightening and epochs size was set to 30. The batch size was set to 8.
7. Finally, the model was trained using the above function and summary was generated.
8. Than we have saved the above model as .pkl file applied 'Pegasus paraphrase' model for rephrasing the summary.

7.1.3 OCR Implementation

To get text from the image uploaded by the user, an **OCR** (Optical Character Recognition) was made using **Pytesseract Library in Python.** There were some **pre-processing steps that were involved** before passing the uploaded image to **Pytesseract Function.** Here are the steps that were performed:

1. Convert the image to grayscale
2. Apply Gaussian blur to reduce noise.
3. Apply thresholding to create a binary image.
4. Normalize the image to a consistent range.
5. Scaling the Image which is to resize the image to double its original size.

After performing these steps, the pre-processed image was passed to **Pytesseract function** and the output text was returned. Then, some **post- processing** steps were applied which are as follow:

1. Remove Multiple Consecutive Spaces
2. Join Lines Ending with Hyphen
3. Remove the break lines

7.2 Test case Design and Description

7.2.1 Verify Signup with valid credentials.

Sigh up			
Test Case ID:	*01*	**QA Test Engineer:**	*Zulqarnain Ishaq*
Test case Version:	*1*	**Reviewed By:**	*Mustafa Khalid*
Test Date:	*26/04/2023*	**Use Case Reference(s):**	*1 (sigh up)*
Revision History:	*None*		
Objective	*To check user can create an account successfully.*		
Product/Ver/Module:	*Web signup module*		
Environment:	*Node js, Django, browser (latest), web is running connected with db.*		
Assumptions:	*Have an active internet connection*		
Pre-Requisite:	*No account exists with this email.*		
Step No.	**Execution description**	**Procedure result**	
1	*User Click on sign up button*	*System displays the signup page.*	
2	*User enters valid email, name, password and click on register button*	*System shows message account created successfully and redirect user to home page.*	
Comments: email format and strong password conditions checked properly and not let person register if not valid, or if password and repeat password not same not let user to register/signup. **Password must be 1 uper,1lower,1digit and 8 or more characters long**			
Passed *Failed* *Not Executed*			

7.2.2 Verify Signup with existing account email.

Sigh up			
Test Case ID:	*02*	**QA Test Engineer:**	*Zulqarnain Ishaq*
Test case Version:	*1*	**Reviewed By:**	*Muhammad Faiq*
Test Date:	*26/04/2023*	**Use Case Reference(s):**	*1 (sigh up)*
Revision History:	*None*		
Objective	*To check user cannot be able to create an account with already existing account's email.*		
Product/Ver/Module:	*Web signup module*		
Environment:	*Node js, Django, browser (latest), web is running connected with db.*		
Assumptions:	*User internet connection is active*		

Pre-Requisite:	*Account already exists with this email enter by user.*	
Step No.	**Execution description**	**Procedure result**
1	*User Click on sign up button*	*System displays the signup page.*
2	*User enters email, name, password and click on register button*	*System show message "Account already exist with this email". Stays on sign up page.*
Comments: Email format and strong password conditions checked properly and not let person register if not valid, or if password and repeat password not same not let user to register/signup. **Password must be 1 uper,1lower,1digit and 8 or more characters long**		
Passed *Failed* *Not Executed*		

7.2.3 Verify Login with valid account email and password

Login			
Test Case ID:	*03*	**QA Test Engineer:**	*Zulqarnain Ishaq*
Test case Version:	*1*	**Reviewed By:**	*Mustafa Khalid*
Test Date:	*26/04/2023*	**Use Case Reference(s):**	*2 (Login)*
Revision History:	*None*		
Objective	*To check user can login with valid email and password.*		
Product/Ver/Module:	*Web Login module*		
Environment:	*Node js, Django, browser (latest), web is running connected with db.*		
Assumptions:	*User internet connection is active*		
Pre-Requisite:	*Account already exists with this email enter by user.*		
Step No.	**Execution description**	**Procedure result**	
1	*User Click on Login button at top bar.*	*System displays the Login page.*	
2	*User enters email, password and click on Login button*	*System show message "Login Successfully "Redirect user to home page.*	
Comments:			
Passed *Failed* *Not Executed*			

7.2.4 Verify Login with invalid account email and password.

Login			
Test Case ID:	*04*	**QA Test Engineer:**	*Zulqarnain Ishaq*
Test case Version:	*1*	**Reviewed By:**	*Mustafa Khalid*
Test Date:	*26/04/2023*	**Use Case Reference(s):**	*2 (Login)*
Revision History:	*None*		
Objective	*To check user cannot be able to login with invalid email and password.*		
Product/Ver/Module:	*Web Login module*		

Environment:	*Node js, Django, browser (latest), web is running connected with db.*	
Assumptions:	*User internet connection is active*	
Pre-Requisite:	*No Account exists with this email enter by user.*	
Step No.	**Execution description**	**Procedure result**
1	*User Click on Login button at top bar.*	*System displays the Login page.*
2	*User enters email, password and click on Login button*	*System show message "Invalid Credentials "Stays at Login page.*
Comments: **Either email is invalid, or password is invalid, or no account exist with this email.**		
☐ *Passed* ☐ *Failed* ☐ *Not Executed*		

7.2.5 Verify get summary by giving text input.

Get summary of text			
Test Case ID:	*05*	**QA Test Engineer:**	*Zulqarnain Ishaq*
Test case Version:	*1*	**Reviewed By:**	*Mustafa Khalid*
Test Date:	*26/04/2023*	**Use Case Reference(s):**	*5 (Summary)*
Revision History:	*None*		
Objective	*To check user can get summary of enter text*		
Product/Ver/Module:	*Web Summary module*		
Environment:	*Node js, Django, browser (latest), web is running connected with db.*		
Assumptions:	*User internet connection is active*		
Pre-Requisite:	*User is on home page of website and Copied news article*		
Step No.	**Execution description**	**Procedure result**	
1	**Users click on summarize button**	**System show message No Text.**	
2	*User Clicks paste button of "Enter News text"*	*Text is displayed at lower text box.*	
3	*User clicks on summarize button.*	*System show message "Please select any model "Stays at Login page.*	
4	*User selects any model and click on summarize button.*	*System first show "Loading... "after few seconds summary and 3 rough scores shown in News Summary box.*	
Comments: **Text must be news text and copy paste button works for the corresponding text box below them.**			
☐ *Passed* ☐ *Failed* ☐ *Not Executed*			

7.2.6 Verify save summary privately.

Save Summary privately			
Test Case ID:	*05*	**QA Test Engineer:**	*Zulqarnain Ishaq*
Test case Version:	*1*	**Reviewed By:**	*Mustafa Khalid*
Test Date:	*26/04/2023*	**Use Case**	*8 (Save Summary)*

		Reference(s):	
Revision History:	*None*		
Objective	*To check user can save the summary privately.*		
Product/Ver/Module:	*Web Summary save module*		
Environment:	*Node js, Django, browser (latest), web is running connected with db.*		
Assumptions:	*User internet connection is active*		
Pre-Requisite:	*User is login and generated summary.*		
Step No.	**Execution description**	**Procedure result**	
1	**Users click on private**	**System will now keep it private .**	
2	*Users click on save summary button.*	*Summary save successfully.*	
Comments: **Private summary will be only visible to owner of summary who generate it and save it.**			
Passed *Failed* *Not Executed*			

7.2.7 Verify save summary publicly.

Save Summary publicly			
Test Case ID:	*06*	**QA Test Engineer:**	*Zulqarnain Ishaq*
Test case Version:	*1*	**Reviewed By:**	*Mustafa Khalid*
Test Date:	*26/04/2023*	**Use Case Reference(s):**	*8 (Save Summary)*
Revision History:	*None*		
Objective	*To check user can save the summary publicly.*		
Product/Ver/Module:	*Web Summary save module*		
Environment:	*Node js, Django, browser (latest), web is running connected with db.*		
Assumptions:	*User internet connection is active*		
Pre-Requisite:	*User is login and generated summary.*		
Step No.	**Execution description**	**Procedure result**	
1	*Users click on save summary button.*	*Summary save successfully.*	
Comments: **Not selecting private option will let summary to be save public and now it will be visible to all.**			
Passed *Failed* *Not Executed*			

7.2.8 Verify rate summary.

Rate Summary			
Test Case ID:	*07*	**QA Test Engineer:**	*Zulqarnain Ishaq*
Test case Version:	*1*	**Reviewed By:**	*Mustafa Khalid*
Test Date:	*26/04/2023*	**Use Case Reference(s):**	*10 (Rate Summary)*
Revision History:	*None*		
Objective	*To check user can rate the summary.*		

Product/Ver/Module:	*Web Summary rate module*	
Environment:	*Node js, Django, browser (latest), web is running connected with db.*	
Assumptions:	*User internet connection is active*	
Pre-Requisite:	*User is login and summary is saved and user has access to it.*	
Step No.	**Execution description**	**Procedure result**
1	*User drags mouse from left to right and click.*	*System refresh and ratting will display at below with the username who rate it.*
Comments: **If user already rate the summary now rate again will update the previous ratting given by user to that summary.**		
☐ *Passed* ☐ *Failed* ☐ *Not Executed*		

7.2.9 Verify search summary

Search Summary			
Test Case ID:	*08*	**QA Test Engineer:**	*Zulqarnain Ishaq*
Test case Version:	*1*	**Reviewed By:**	*Mustafa Khalid*
Test Date:	*26/04/2023*	**Use Case Reference(s):**	*12 (Search Summary)*
Revision History:	*None*		
Objective	*To check user can search the summary from api used.*		
Product/Ver/Module:	*Web Summary Search module*		
Environment:	*Node js, Django, browser (latest), web is running connected with db.*		
Assumptions:	*User internet connection is active*		
Pre-Requisite:	*User internet active.*		
Step No.	**Execution description**	**Procedure result**	
1	*User clicks on search News article button at top right.*	*System will show the search summary page.*	
2	*User moves mouse near the search option.*	*System will open search bar.*	
3	*User enters text in the search bar and click on search button.*	*System show "Loading..." tag then after few seconds all relevant articles shown below if exist in the api set of articles. If no relevant found it will show message that none exist.*	
Comments: **If none of article exist in api then it will show that nonrelevant found from api search.**			
☐ *Passed* ☐ *Failed* ☐ *Not Executed*			

7.2.10 Verify change privacy of summary.

Change privacy of Summary			
Test Case ID:	*09*	**QA Test Engineer:**	*Zulqarnain Ishaq*
Test case Version:	*1*	**Reviewed By:**	*Mustafa Khalid*
Test Date:	*26/04/2023*	**Use Case Reference(s):**	*11 (change privacy of Summary)*
Revision History:	*Save summary*		

Objective	*To check user can change privacy of summary.*	
Product/Ver/Module:	*Web Summary privacy module*	
Environment:	*Node js, Django, browser (latest), web is running connected with db.*	
Assumptions:	*User internet connection is active.*	
Pre-Requisite:	*User is login and summary are saved and user has ownership to it.*	
Step No.	**Execution description**	**Procedure result**
1	*User will click on button "Change to public" or "change to private".*	*The system will show change to public if saved summary is private and vice versa before that. System will show message of "changed the privacy" and redirect to home page.*
Comments: **User can change public to private and vice versa if he/she has ownership of that summary.**		
☐ *Passed* ☐ *Failed* ☐ *Not Executed*		

7.2.11 Verify delete summary.

Delete Summary			
Test Case ID:	*10*	**QA Test Engineer:**	*Zulqarnain Ishaq*
Test case Version:	*1*	**Reviewed By:**	*Mustafa Khalid*
Test Date:	*26/04/2023*	**Use Case Reference(s):**	*13 (Delete Summary)*
Revision History:	*Saved Summary*		
Objective	*To check user can delete the summary.*		
Product/Ver/Module:	*Web Summary rate module*		
Environment:	*Node js, Django, browser (latest), web is running connected with db.*		
Assumptions:	*User internet connection is active*		
Pre-Requisite:	*User is login and summary are saved and user has ownership to it.*		
Step No.	**Execution description**	**Procedure result**	
1	*User clicks on delete button.*	*System show message "deleted successfully" and redirect to home page.*	
Comments: **User can only delete summary if he/she is owner of summary.**			
☐ *Passed* ☐ *Failed* ☐ *Not Executed*			

7.2.12 Verify show all save summaries (View history of summaries).

Show all Summaries			
Test Case ID:	*11*	**QA Test Engineer:**	*Zulqarnain Ishaq*
Test case Version:	*1*	**Reviewed By:**	*Mustafa Khalid*
Test Date:	*26/04/2023*	**Use Case Reference(s):**	*9 (View Summary history)*
Revision History:	*Saved Summary*		
Objective	*To check user can view the history of all summaries.*		
Product/Ver/Module:	*Web Summary history module*		

Environment:	*Node js, Django, browser (latest), web is running connected with db.*	
Assumptions:	*User internet connection is active*	
Pre-Requisite:	*User has access right to that summary.*	
Step No.	**Execution description**	**Procedure result**
1	*User clicks on green + button at home page.*	*System shows all public summaries.*
2	*User click on show my all summaries in his/her profile.*	*System shows all summaries whose owner that user is.*
Comments: **Access right means either summary is public can access by anyone or private and is accessed by owner of that summary.**		
☐ *Passed* ☐ *Failed* ☐ *Not Executed*		

7.2.13 Verify user can edit profile.

Edit profile			
Test Case ID:	*12*	**QA Test Engineer:**	*Zulqarnain Ishaq*
Test case Version:	*1*	**Reviewed By:**	*Mustafa Khalid*
Test Date:	*26/04/2023*	**Use Case Reference(s):**	*3 (Edit profile)*
Revision History:	*Sign up /Registered*		
Objective	*To check user can edit his/her profile.*		
Product/Ver/Module:	*Web edits profile module*		
Environment:	*Node js, Django, browser (latest), web is running connected with db.*		
Assumptions:	*User internet connection is active*		
Pre-Requisite:	*User has sign up /register (has account).*		
Step No.	**Execution description**	**Procedure result**	
1	*User clicks on edit profile button.*	*System shows edit profile page.*	
2	*User enters updated changes and click save changes.*	*System will show the updates on edit profile page.*	
Comments: **At place text username is displayed.**			
☐ *Passed* ☐ *Failed* ☐ *Not Executed*			

7.2.14 Verify forget password recovery.

Forget password recovery			
Test Case ID:	*13*	**QA Test Engineer:**	*Zulqarnain Ishaq*
Test case Version:	*1*	**Reviewed By:**	*Mustafa Khalid*
Test Date:	*26/04/2023*	**Use Case Reference(s):**	*4 (Reset forget password)*
Revision History:	*Sign up /Registered*		
Objective	*To check user, recover the forgotten password.*		
Product/Ver/Module:	*Web account password recovery module*		
Environment:	*Node js, Django, browser (latest), web is running connected with db.*		
Assumptions:	*User internet connection is active*		
Pre-Requisite:	*User has sign up /register (has account).*		

Step No.	Execution description	Procedure result
1	*User clicks on forget password option on login page.*	*System show page to enter email.*
2	*User enters email and press send mail button.*	*System will send an email to user after checking that it's valid. Email will have reset password page link.*
3	*Use clicks on link available in reset email.*	*Move to reset password page.*
4	*User enters password, retype it in retype field. Hit saved button.*	*User password will be change and user will be re-directed to login page.*
Comments: **if password and repeat password not same not let user to save password.** **Password must be 1 uper,1lower,1digit and 8 or more characters long**		
☐ *Passed* ☐ *Failed* ☐ *Not Executed*		

7.3 Test Metrics

7.3.1 Sample Test case Matric No.1

Metric	Purpose	
Number of Test Cases	10	
Number of Test Cases Passed	10	
Number of Test Cases Failed	0	
Test Case Defect Density	0	
Test Case Effectiveness	100%	
Traceability Matrix	**Requirement Description**	**Test Case Description**
	The system shall allow the user to sign up with valid credentials.	Verify Signup with valid credentials.

7.3.2 Sample Test case Matric No.2

Metric	Purpose	
Number of Test Cases	10	
Number of Test Cases Passed	10	
Number of Test Cases Failed	0	
Test Case Defect Density	0	
Test Case Effectiveness	100%	
Traceability Matrix	**Requirement Description**	**Test Case Description**
	The system shall allow the user to sign up with existing account email.	Verify Signup with existing account email.

7.3.3 Sample Test case Matric No.3

Metric	Purpose
Number of Test Cases	10
Number of Test Cases Passed	10
Number of Test Cases Failed	0

Test Case Defect Density	0	
Test Case Effectiveness	100%	
Traceability Matrix	**Requirement Description**	**Test Case Description**
	The system shall allow the user to login with valid account email & password.	Verify Login with valid account email and password

7.3.4 Sample Test case Matric No.4

Metric	**Purpose**	
Number of Test Cases	10	
Number of Test Cases Passed	10	
Number of Test Cases Failed	0	
Test Case Defect Density	0	
Test Case Effectiveness	100%	
Traceability Matrix	**Requirement Description**	**Test Case Description**
	The system shall not allow the user to login with invalid account email and password.	Verify Login with invalid account email and password.

7.3.5 Sample Test case Matric No.5

Metric	**Purpose**	
Number of Test Cases	10	
Number of Test Cases Passed	10	
Number of Test Cases Failed	0	
Test Case Defect Density	0	
Test Case Effectiveness	100%	
Traceability Matrix	**Requirement Description**	**Test Case Description**
	The system shall allow the user to get summary by giving text input.	Verify get summary by giving text input.

7.3.6 Sample Test case Matric No.6

Metric	**Purpose**	
Number of Test Cases	10	
Number of Test Cases Passed	10	
Number of Test Cases Failed	0	
Test Case Defect Density	0	
Test Case Effectiveness	100%	
Traceability Matrix	**Requirement Description**	**Test Case Description**
	The system shall allow the user to save summary privately.	Verify save summary privately.

7.3.7 Sample Test case Matric No.7

Metric	Purpose	
Number of Test Cases	10	
Number of Test Cases Passed	10	
Number of Test Cases Failed	0	
Test Case Defect Density	0	
Test Case Effectiveness	100%	
Traceability Matrix	Requirement Description	Test Case Description
	The system shall allow the user to save summary publicly.	Verify save summary publicly.

7.3.8 Sample Test case Matric No.8

Metric	Purpose	
Number of Test Cases	10	
Number of Test Cases Passed	10	
Number of Test Cases Failed	0	
Test Case Defect Density	0	
Test Case Effectiveness	100%	
Traceability Matrix	Requirement Description	Test Case Description
	The system shall allow the user to rate summary.	Verify rate summary.

7.3.9 Sample Test case Matric No.9

Metric	Purpose	
Number of Test Cases	10	
Number of Test Cases Passed	10	
Number of Test Cases Failed	0	
Test Case Defect Density	0	
Test Case Effectiveness	100%	
Traceability Matrix	Requirement Description	Test Case Description
	The system shall allow the user to search summary.	Verify search summary.

7.3.10 Sample Test case Matric No.10

Metric	Purpose	
Number of Test Cases	10	
Number of Test Cases Passed	10	
Number of Test Cases Failed	0	
Test Case Defect Density	0	
Test Case Effectiveness	100%	
Traceability Matrix	Requirement Description	Test Case Description

	The system shall allow the user to change privacy of summary.	Verify change privacy of summary.

7.3.11 Sample Test case Matric No. 11

Metric	**Purpose**	
Number of Test Cases	10	
Number of Test Cases Passed	10	
Number of Test Cases Failed	0	
Test Case Defect Density	0	
Test Case Effectiveness	100%	
Traceability Matrix	**Requirement Description**	**Test Case Description**
	The system shall allow the user to delete summary.	Verify delete summary.

7.3.12 Sample Test case Matric No.12

Metric	**Purpose**	
Number of Test Cases	10	
Number of Test Cases Passed	10	
Number of Test Cases Failed	0	
Test Case Defect Density	0	
Test Case Effectiveness	100%	
Traceability Matrix	**Requirement Description**	**Test Case Description**
	The system shall allow the user to show all save summaries.	Verify show all save summaries (View history of summaries).

7.3.13 Sample Test case Matric No.13

Metric	**Purpose**	
Number of Test Cases	10	
Number of Test Cases Passed	10	
Number of Test Cases Failed	0	
Test Case Defect Density	0	
Test Case Effectiveness	100%	
Traceability Matrix	**Requirement Description**	**Test Case Description**
	The system shall allow the user to edit profile.	Verify user can edit profile.

7.3.14 Sample Test case Matric No.14

Metric	**Purpose**
Number of Test Cases	10
Number of Test Cases Passed	10
Number of Test Cases Failed	0

Test Case Defect Density	0	
Test Case Effectiveness	100%	
Traceability Matrix	**Requirement Description**	**Test Case Description**
	The system shall allow the user to recover forget password.	Verify forget password recovery.

Chapter 8: User Manual

Our ultimate product is a news text summarizer which can summarize news text articles by using different deep learning models. To help the user install the setup on their system and to generate summary, here are few steps to follow:

8.1 Install Node.js on System

Firstly, the user shall install Node.js on his/her system by visiting their official website. Than the user shall open, the terminal and put the following command: **npm start**. After this command, the user shall be prompted to the website of our system.

8.2 Register an account using the Signup Page

Now, the user have to sign up and create an account by mentioning his/her name, email id and password. The account shall be created successfully and the user shall be prompted to the login page.

8.3 Login into your Account

In the next step, the user shall login to his/her account by mentioning the correct email id and password. The user shall be prompted to the main page where he/she can generate summaries.

8.4 Enter Text

Next, the user shall paste the desired text in the “Enter News Text ”Box by clicking on the paste button.

8.5 Generate Summary

Next, the user shall select one of the machine learning models, and then press on the summarize button. The summary shall be generated successfully!

8.6 Save Summary

The user can also rate the given summary from 1 star to 5 star by pressing on the number of starts and clicking on rate button.

8.7 Rate Summary

The user can also rate the summary either publicly or privately by clicking on the save summary button.

Chapter 9: Experimental Results and Discussion

We evaluated our models using rogue scores and following were the results obtained, the epochs decided for training were selected to ensure best performance for each model using hit and trial:

9.1 Recurrent Neural Networks

9.1.1 Simple RNNs with embeddings

As seen in the below table, RNN based encoder decoders with pretrained embeddings almost generate nil summaries and the change in embeddings hardly exhibit any influence on the model to learn good contextual relationships between input summaries and input texts.

Table 4: Summaries Generated by RNN Models

The table shows the original article and the respective summary of that article generated by various RNNs deep learning models.

No.	Model Used	Article	Summary Generated
1.	RNN LSTM based(Glove)	"Ever noticed how plane seats appear to be getting smaller and smaller? With increasing numbers of people taking to the skies, some experts are questioning if having such packed out planes is putting passengers at risk. They say that the shrinking space on aero planes is not only uncomfortable - it's putting our health and safety in danger……."	"the airline has created a new York based on the air force to create a new York based on the flight from the air force the faa has been working with a new York based on the flight"
2.	RNN LSTM(Word 2Vec)	"Ever noticed how plane seats appear to be getting smaller and smaller? With increasing numbers of people taking to the skies, some experts are questioning if having such packed out planes is putting passengers at risk. They say that the shrinking space on aero planes is not only uncomfortable - it's putting our health and safety in danger……..."	"the faa says the ban is not a problem for the first time the airline says the plane is not a flight attendants of the plane the plane is a size of the size of the size of the size of the size of the size of the size of the size of the size of the size of the size of the size of the country."
3.	RNN LSTM(Learnt Embeddings)	"Ever noticed how plane seats appear to be getting smaller and smaller? With increasing numbers of people taking to the skies, some experts are questioning if having such packed out planes is putting passengers at risk. They say that	"the airline s airline has been able to get a new airlines flight to the u s to the u s and the us are not able to get the airline to help the airline s airline says it is not a problem of the airline s airline"

		the shrinking space on aero planes is not only uncomfortable - it's putting our health and safety in danger.”	

To give a more objective summary of the results, following were the rogue scores generated on the dataset of CNN/Daily Mail:

Table 5: Rouge Scores of RNN Models

The table shows the rouge scores of each RNN model including the dataset size as well.

Model used	R1 Measure	R2 Measure	RL Measure	Training time	Dataset size
RNN LSTM(Glove)	0.14	0.018	0.13	3 hours	110,000
RNN LSTM(Word2Vec)	0.145	0.017	0.136	3 hours	110,000
RNN LSTM(Learn Embeddings)	0.147	0.021	0.137	3 hours	110,000

9.1.2 Attention Based RNNs

To enhance our experiments, we tried RNNs with an additional attention mechanism to observe the results. Furthermore, in above experiments, it can be seen that none of our RNN based models resulted in adequate summaries. We conducted a brief series of experiments using teacher forcing mechanism, with different batch sizes. We notice that by adding the attention mechanism, we were able to increase the sensibility of output sentences and having somewhat more coherent grammar than the non-attention counter part of the same model.

Furthermore, we also included a [UNK] token in order to reduce non sense words being part of the summary output and instead output an OOV word instead. While investigating the behavior of this model, we encountered a number of problems such as:

Excessive OOV words generation

It is a very common problem in many language generation tasks. In order to lessen [UNK] token generation, we tried increasing the size of vocabulary. However, increasing the size of vocabulary also meant reducing masked accuracy of the model which was a tradeoff. So we also tried to change the batch size to see any difference.

Table 6: Excessive OOV words generation

The table shows different parameters for Excessive OOV words generation such as batch size, rouge scores, etc

Batch size	Vocabulary Size	Epochs	Masked accuracy	Rogue 1	Rogue 2	Rogue L
16	5000	30	32%	7.5%	1%	7.2%
16	7500	30	30.19%	10.27%	1.15%	9.6%
16	10,000	30	28.69%	12.01%	1.5%	11.43%
32	5000	30	43%	16.1%	3.14%	14.83%
32	7500	30	42.9%	17.02%	4.9%	15.9%

32	10,000	30	41.68%	19.47%	4.9%	18.42%
64	5000	30	45.36%	16.49%	3.86%	15.76%
64	7500	30	45.3%	17.23%	4.2%	16.25%
64	10,000	Google Colab Crashed	NA	NA	NA	NA

Furthermore, we also experimented using an LSTM based RNN to notice any difference. We picked the following parameters to see if any difference can be observed: Batch size 32, 64. Vocabulary size: 5000,7500. Following were the issues encountered when experimenting:

Exploding gradient problem

We noticed that with LSTMs, increasing vocabulary resulted in the model dropping its masked accuracy severely after a set number of epochs. This cut off number was 16 as we increased the batch size. To solve this problem, we tried gradient clipping and our results seemed to stabilized but with poorer accuracies than GRU model. With gradient clipping, results were stabilized and batch size 64 gave better results as shown in the table below:

Table 7: Results without gradient clipping [1] and with gradient clipping [2]

The table shows results of GRU model without gradient clipping

Batch size	Vocabulary Size	Epochs	Masked Accuracy	Rogue 1	Rogue 2	Rogue L
32 [1]	5000	30	42%	11.65%	3.2%	11.02%
32 [1]	7500	16(after 16, drop was experienced)	37%	11.3%	1.9%	10.73%
64 [1]	5000	16(after which the accuracy drops drastically)	34%	11.09%	0.97%	10.25%
32 [2]	7500	30	27%	10%	0.7	9.8%
64 [2]	5000	30	44.18%	13.57%	3.61%	13.04%

A sample summary generated by the best RNN model which is a GRU with batch size 32 vocabulary size of 10,000:

Table 8: Summaries Generated by GRU Model
The table shows the original article and the respective summary of that article generated by GRU model.

Input Text	Summary Generated
A drunk driver who killed a young woman in a head-on crash while checking his mobile phone has been jailed for six years. Craig Eccleston-Todd, 27, was driving home from a night at a pub when he received a text message. As he was reading or replying to it, he veered across the road while driving round a bend and smashed into Rachel Titley's car coming the other way. Craig Eccleston-Todd, 27 (left) was using his mobile phone when he crashed head-on into the car being driven by Rachel Titley, 28 (right). She died later from her injuries"	craig [UNK] , , was driving home from a pub when he crashed into the car . he was found guilty of causing death by dangerous driving . [UNK] was found guilty of causing death by dangerous driving

Test on our dataset

Furthermore, we also tried training this model on our created dataset which contains technical content and only close to 200 training tuples. While it performs well on the technical data set and produces good summaries, having a good training score but fails to generalize on unseen and cross domain data and only outputs the content it has memorized during training regardless of context. This is most likely due to overfitting since the model is being trained from scratch and has only been exposed to a handful number of training examples. Following were the rogue scores seen on the technical data:

Table 9: Rouge Scores of GRU Model
The table shows the rouge scores of each GRU model.

R1 Score	R2 Score	RL Score
0.51	0.41	0.50

And following is the sample summary generated on this dataset:

Table 10: Summaries Generated on Dataset
The table shows the original article and the respective summary of that article generated by GRU model on dataset.

Input Text	Summary Generated
Ultimately, a written text is composed of characters—letters, digits, punctuation, and spaces in English (and more exotic characters in some other languages). Thus, one of the simplest language models is a probability distribution over sequences of characters. As in Chapter 15, we write P (c1:N) for the probability of a sequence of N characters, c1 through cN . In one Web collection, P ("the")	a written text is made up of characters , so a simple language model is a probability distribution over sequences of characters . probability of sequences of n characters can be written as p cn . sequences of written symbols of length n are called ngrams'

= 0.027 and P (“zgq”) = 0.000000002. A sequence of written sym- bols of length n is called an n-gram (from the Greek root for writing or letters), with special case “unigram” for 1-gram, “bigram” for 2-gram, and “trigram” for 3-gram. A model of the probability distribution of n-letter sequences is thus called an n-gram model. (But be care- ful: we can have n-gram models over sequences of words, syllables, or other units; not just over characters.	

However, on cross domain input, it seems to generate completely unplausible results and irrelevant summaries even if the context doesn’t belong to the technical domain and seems to output a summary that’s similar to length of summaries in the training context:

Table 11: Summaries Generated on cross domain input

The table shows the original article and the respective summary of that article generated by GRU model on cross domain input.

Input Text	Summary Generated
News reporters are lonely people and they should work on themselves. It has achieved world peace. He was then taken to hospital but died later from her injuries	the complexity of the relationship between an agents behavior , the percept sequence generated by the environment , and the performance measure is what matters , not whether the environment is real or artificial.

9.2 Transformers

Three state of the art transformer models were used to produce summaries and do experimentation of producing abstractive summaries. The models used were SimpleT5, Hybrid Model (Combination of T5 and Pegasus Model), and BART model. The sample summaries produced by each model is shown in the table below:

Table 12: Summaries Generated by RNN Models

The table shows the original article and the respective summary of that article generated by various transformer models.

No.	Model Used	Article	Summary Generated
1.	SimpleT5 Model	“Craig Eccleston-Todd, a drunk driver who killed a young woman in a head-on crash while checking his mobile phone, has been sentenced to six years in jail. The accident happened in 2013 when Eccleston-Todd was driving home from a night at the pub, and received a text message. As he read or replied to the message, he veered across the road, crashing into Rachel Titley’s car coming the other way. Titley, a 28-year-old solicitor’s clerk from	“Craig Eccleston-Todd was driving home from a pub when he crashed. He had text message and then veered across the road, crashing into Rachel Titley's car. The 28-year-old solicitor's clerk later died from her injuries in hospital. Eccleston-Todd has been sentenced to six years in jail for dangerous driving.”

		Cowes, Isle of Wight, had not consumed any alcohol that evening, and had been driving responsibly. She was unable to avoid the collision, and later died from her injuries in hospital. Eccleston-Todd, found guilty of causing death by dangerous driving, had drunk at least three or four pints of beer before getting behind the wheel. He was texting around the time of the crash and may have also been over the drink-drive limit.”	
2.	Hybrid Model	“Craig Eccleston-Todd, a drunk driver who killed a young woman in a head-on crash while checking his mobile phone, has been sentenced to six years in jail. The accident happened in 2013 when Eccleston-Todd was driving home from a night at the pub, and received a text message. As he read or replied to the message, he veered across the road, crashing into Rachel Titley’s car coming the other way. Titley, a 28-year-old solicitor’s clerk from Cowes, Isle of Wight, had not consumed any alcohol that evening, and had been driving responsibly. She was unable to avoid the collision, and later died from her injuries in hospital. Eccleston-Todd, found guilty of causing death by dangerous driving, had drunk at least three or four pints of beer before getting behind the wheel. He was texting around the time of the crash and may have also been over the drink-drive limit.”	“A drunk driver who killed a woman in a head-on crash while checking his phone was sentenced to six years in jail. The driver may have been over the drink-drive limit because he had drunk at least three or four beers before getting behind the wheel.
3.	BART Model	“Craig Eccleston-Todd, a drunk driver who killed a young woman in a head-on crash while checking his mobile phone, has been sentenced to six years in jail. The accident happened in 2013 when Eccleston-Todd was driving home from a night at the pub, and received a text message. As he read or replied to the message, he veered across the road, crashing into Rachel Titley’s car coming the other way. Titley, a 28-year-old solicitor’s clerk from Cowes, Isle of Wight, had not	“"Craig Eccleston-Todd, a drunk driver who killed a young woman. The accident happened in 2013 when Eccleston-Todd was driving home from a night. Trippley has just been charged with theft after a crash in central Sydney.\n Craig Titley, a 28-year-old solicitor’s clerk from Cow She was unable to avoid the collision, and later died at the scene.\nAn Todd: Bailey Eccleston-Todd, found guilty of causing death by dangerous

		consumed any alcohol that evening, and had been driving responsibly. She was unable to avoid the collision, and later died from her injuries in hospital. Eccleston-Todd, found guilty of causing death by dangerous driving, had drunk at least three or four pints of beer before getting behind the wheel. He was texting around the time of the crash and may have also been over the drink-drive limit.”	driving. Man was texting around the time of the crash and may have also been over the drink.'”

The rouge scores of the each transformer model is given below. The dataset used for training the models was from CNN/Daily-Mail news articles. The dataset size was kept equal and standard which is 20,000. Each Model was trained for 3 epochs due to limited amount of resources. The rouge scores suggest that SimpleT5 Model produced the best rouge score results compared to other models. However, BART model also produced comparatively same Rouge Scores. SimpleT5 model also produced good rouge scores compared to other kind of encoder-decoder models used previously in the experimentation.

Table 13: Rouge Scores of Transformer Models
The table shows the rouge scores of each transformer model including the dataset size as well.

Model used	R1 Measure	R2 Measure	RL Measure	Training time	Dataset size
SimpleT5 Model	0.372	0.16	0.356	3 hours	20,000
Hybrid Model	0.327	0.11	0.304	3 hours	20,000
BART Model	0.371	0.12	0.338	4 hours	20,000

The results from these experimentation suggest that the SimpleT5 model is the best model out of all the three models for producing semantically correct abstractive summaries. BART model has also showed quite good rouge scores but the summaries produced by BART model were having some grammatical mistakes and were skipping important information sometimes. By evaluating the summaries of each model, SimpleT5 seems to produce the best summaries out of all the three models. For further clarification about the evaluation of summaries produced by SimpleT5 and Hybrid model, we performed human evaluation in which we took feedback from humans about summaries. We have only used SimpleT5 and Hybrid model as they were producing semantically correct summaries. The detailed results about human evaluation will be given in the “Human Evaluation” section.

Test on our dataset

Furthermore, we also tried training SimpleT5 and Hybrid model on our created dataset which contains technical content and only close to 200 training tuples. The rouge scores produced by these models on technical dataset is given below:

Table 14: Rouge Scores of Transformer Models on Technical Dataset
The table shows the rouge scores of each transformer model including the technical dataset size as well.

Model used	R1 Measure	R2 Measure	RL Measure	Training time	Technical Dataset size
SimpleT5 Model	0.586	0.388	0.55	30 mins	200
Hybrid Model	0.512	0.312	0.462	30 mins	200

The rouge scores shows that simpleT5 and hybrid model trained on technical dataset outperforms all other transformer models in terms of rouge scores. This suggest us that, our technical dataset is suitable for training transformer models for producing good abstractive summaries with high rouge scores. A sample summary produced by SimpleT5 model being trained on technical dataset is given below:

Table 15: Summary Generated by SimpleT5

The table shows the original article and the respective summary of that article generated by SimpleT5 model on technical dataset.

Input Text	Summary Generated
Formal languages, such as the programming languages Java or Python, have precisely defined language models. A language can be defined as a set of strings; ,Äúprint(2 + 2),Äù is a legal program in the language Python, whereas ,Äú2)+(2 print,Äù is not. Since there are an infinite number of legal programs, they cannot be enumerated; instead they are specified by a set of rules called a grammar. Formal languages also have rules that define the meaning or semantics of a program; for example, the rules say that the ,Äúmeaning,Äù of ,Äú2 + 2,Äù is 4, and the meaning of ,Äú1/0,Äù is that an error is signaled.	Formal languages, such as Java or Python, have defined language models. A language can be defined as a set of strings, while legal programs are specified by a set of rules called a grammar. Formal languages also have rules that define the meaning or semantics of a program, such as the meaning of â2 + 2â being 4, and the meaning of â1/0â signaling an error.

9.3 Human Evaluation

Since language is a complex amalgamation of different syntactic and semantic elements, it's very difficult to provide a good or concrete evaluation through automatic evaluation methods alone. For abstractive text summarization especially, this is a difficult task and there are no known evaluation metrics which capture all linguistic complications available in language.

Hence, for this purpose we employed human evaluation methodology which involved 36 participants, asked to rate our summaries on a linear scale of 1 to 5. For evaluation purposes, we used two of our best models, **SimpleT5 and Hybrid** which seemed to produce good rogue scores making them suitable candidates for human evaluation. We picked 5 input texts, each having 2 generated summaries, belonging to a model. A sample input text with two generated summaries by each model is given below:

Table 16: Sample Summary Generated by Models

The table shows the original article and the respective summary of that article generated by SimpleT5 and Hybrid Model.

Input Text	Summary Generated

President Putin annexed Crimea to Russian domain in a swift move, but imposing sanctions on Russia for this land grab won't be as easy for Europe. Initially, Europe put visa restrictions and asset freezes on a few politicians and military personnel, but there are calls to target the Russian business community. EU leaders must be ready for the consequences of sanctions as they are a two-way street. Targeting Russia's business community could hurt their support for President Putin and help Europe deal with corruption. However, Europe should not risk trade, as Russia provides gas for Europe, which is essential for keeping factories running and citizens warm. Open trade channels allow dialogue to continue, and no one can afford to cut off that lifeline, especially with Europe's economy on the rebound and Russia's economy on the wane.	**(By Hybrid Model)** EU leaders need to be aware of the consequences of sanctions. Targeting Russia's business community could hurt their support for President Putin. Russia provides gas for Europe, which is essential for keeping factories running and citizens warm.
President Putin annexed Crimea to Russian domain in a swift move, but imposing sanctions on Russia for this land grab won't be as easy for Europe. Initially, Europe put visa restrictions and asset freezes on a few politicians and military personnel, but there are calls to target the Russian business community. EU leaders must be ready for the consequences of sanctions as they are a two-way street. Targeting Russia's business community could hurt their support for President Putin and help Europe deal with corruption. However, Europe should not risk trade, as Russia provides gas for Europe, which is essential for keeping factories running and citizens warm. Open trade channels allow dialogue to continue, and no one can afford to cut off that lifeline, especially with Europe's economy on the rebound and Russia's economy on the wane.	**(By SimpleT5 Model)** Putin annexed Crimea to Russia domain in a swift move, writes Vladimir Smolinsky. He says EU leaders must be ready for the consequences of sanctions as they are a two-way street. Europe's economy is on the rebound and Russia's economy is on the wane.

The results of the five summaries produced by Hybrid Model according to the human evaluation is given below. “Rating 1” is the worst rating while “Rating 5” is the best rating that can be given to the summary by humans:

Table 17: Results Produced by Hybrid Human Feedback

The table shows the results produced by human feedback for Hybrid Model.

Input Text No.	Rating 1	Rating 2	Rating 3	Rating 4	Rating 5
1	0%	13.9%	50%	13.9%	22.2%
2	11.1%	13.9%	30.6%	25%	19.4%
3	8.3%	8.3%	25%	22.2%	36.1%
4	0%	11.1%	36.1%	16.7%	36.1%

5	0%	16.7%	38.9%	19.4%	25%

Similarly, the results of the five summaries produced by SimpleT5 Model according to the human evaluation is given below. “Rating 1” is the worst rating while “Rating 5” is the best rating that can be given to the summary by humans:

Table 18: Results Produced by SimpleT5 Human Feedback
The table shows the results produced by human feedback for SimpleT5 Model.

Input Text No.	Rating 1	Rating 2	Rating 3	Rating 4	Rating 5
1	5.6%	8.3%	36.1%	25%	25%
2	0%	13.9%	27.8%	13.9%	44.4%
3	0%	5.6%	22.2%	33.3%	38.9%
4	2.8%	19.4%	41.7%	16.7%	19.4%
5	0%	8.3%	22.2%	19.4%	50%

The results suggest, out of five summaries, summaries produced by SimpleT5 model was able to achieved more 5 star ratings compared to Hybrid model. Only in input text 4, Hybrid model has more 5 star rating as compared to SimpleT5 model. This analysis concludes that simpleT5 model outperform the remaining models in terms of rouge scores as well as human evaluation.

9.4 Multitask learning

Other than just testing our baseline models on our created dataset, we also touched upon the aspect of multitask learning by leveraging the 3 things that our dataset provides us: First is the human annotated topics specified for each input text, an extractive summary and finally our end to end goal, an abstractive summary.

We created two settings for our experiments: One where we preprocess our dataset in a way it learns to generate abstractive and extractive summaries, another where the model learns to generate both topics and abstractive summaries. This is achieved by prepending a control signal to input text in order to provide the model its required learning hints. This has already been achieved in image processing tasks, we aimed to do the same in our sequence to sequence using the process described above. For training, we create two versions of our dataset and test them separately.

In 4 epochs, the models seemed to converge and produced the following outputs on a sample input text:

Input Text: “Abstractive: Formal languages, such as the programming languages Java or Python, have precisely defined language models. A language can be defined as a set of strings; “print(2 + 2)” is a legal program in the language Python, whereas “2)+(2 print” is not. Since there are an infinite number of legal programs, they cannot be enumerated; instead they are specified by a set of rules called a grammar. Formal languages also have rules that define the meaning or semantics of a program; for example, the rules say that the “meaning” of “2 + 2” is 4, and the meaning of “1/0” is that an error is signaled.”

Where “Abstractive:” is a control signal prepended to input text.

9.4.1 Abstractive-Extractive Model

AbstractiveExtractive model generates the following summary:

Summary: Formal languages have defined language models. A language can be defined as a set of strings; legal programs are specified by a set of rules called a grammar. Formal languages also have rules that define the meaning or semantics of a program; for example, the meaning of “2 + 2” is 4 and the meaning of “1/0” is that an error is signaled.

Observation: Observe that the summary produced is somewhere between abstractive and extractive since the model learns to generate both at the same time. However this isn’t necessarily a bad thing. It means that we are able to take a stronger assumption of factual consistency while managing to introduce creative elements of language.

9.4.2 Topic Guided Model

On the other hand, Topic Guided model generates the following summary:

Summary: Formal languages have precisely defined language models, such as Java or Python. They are specified by a set of rules called a grammar, which defines the meaning or semantics of a program. For example, the rules say that the “meaning” of “2 + 2” is 4 and the meaning of “1/0” indicates that an error is signaled.

Observation: As it can be seen, the topic guided summarization model generates more abstractive summary and tends to be more creative however it is less factually consistent and may join pieces of information with proper rules of linguistic elements without considering their actual relevance. Consider the first sentence for example: Java and python are defined as language models according to the sentence or they might be examples of such language models. It isn’t being made clear.

9.4.3 Test on CNN/Daily Mail

We also tested these model on a CNN/Daily Mail piece of text with an addition of semantic noise. In other words, information that has nothing to with the context of text.

Input Text: "Manchester City moves step closer to meeting European football's financial fair play requirements. The English club posted a deficit of almost $160 million for last season. The latest figure represents the fourth highest deficit in English football history. City's £197.5 million budget"

Noise: The noise included in this text is the last fact regarding City’s 197.5 million budget which is from a completely different input text.

Summary by Topic Guided: Manchester City's deficit for last season was £197.5 million, the fourth highest in English football.

Summary by AbstractiveExtractive Model: Manchester City's deficit of almost $160 million for last season is the fourth highest in English football history.

Observation: As it can be seen, the topic guided summary is definitely abstractive but seems to incorporate this irrelevant piece of information. Whereas AbstractiveExtractive summary not only manages to nail down the entire context but also seems to produce a fairly abstractive summary taking care of the required linguistic elements.

Following were also the rogue scores generated by the AbstractiveExtractive model and topic guided on our technical dataset respectively:

1. R1: 0.69 R2: 0.56 RL: 0.68

2. R1: 0.71 R2: 0.53 RL: 0.69

However, for a fair comparison we also tested this model on CNN/Daily Mail producing the following results which are quite plausible considering that CNN/Daily Mail is a dataset that belongs to an entirely different domain from our technical dataset:

1. **Abstractive-Extractive:** R1: 0.32 R2:0.12 RL: 0.29
2. **Topic Guided:** R1: 0.32 R2: 0.10 RL: 0.30

Chapter 10: Conclusion and Future Work

In this document, we have effectively described and conveyed the problem domain of our proposed research, and managed to come up with an effective methodology to compliment the study.

10.1 Summary

Throughout our study conducted so far, we were able to conduct a comprehensive literature review, letting us grasp a better understanding of the problem domain and what has already been done to solve it. Firstly, we give a high level overview of proposed models that can be used to solve the problem by proposing an effective methodology and were able to conduct experiments on our proposed methodology to demonstrate our findings.

Particularly, we were able to conduct experiments on different flavors of RNN models by using different word embeddings and in the form of a translation model, and evaluated their effectiveness for the task of abstractive summary generation. We also drew a comparative analysis of each of these flavors and also compared them with the transformer model. Turns out, transformer performs the best out of all the models we developed, delivering identical to state of the art results.

10.2 Conclusion

From our study, the following conclusions can be drawn:

1. RNNs alone are highly unsuitable for long sequence to sequence generation tasks since the context vector generated by the RNNs is insufficient to pack the information required from such long walls of text to be fed to the decoder and let it generate adequate summaries. To solve this problem, we tried attention mechanism to focus on various parts of the input text and provides this as information in addition to the information packed in the context vector.
2. SimpleT5 are much more suitable for mapping long sequence contextual dependencies and also manages to draw accurate conceptual relations between texts and generated summaries.
3. Hybrid Transformer model managed to generate very context rich summaries but compromised on the creativity and fluidity of expression.
4. BART model was able to produced summaries with high rouge scores but however, the summaries produced were having grammatical and sometimes semantical issues as well.
5. Despite very limited dataset, it can be seen that our proposed methodology of using multitask learning was able to introduce very interesting results. However, due to this limitation, it also couldn't generate rogue scores as good simplet5 on the same dataset trained on a larger number of examples. However, the scores are a good starting point and can pave way for a future research endeavour.

10.3 Future Work

For future work, we suggest that our multitasking approach be tried on other different kinds of sequence to sequence tasks and create a dataset with larger training examples to familiarize the model with the generative task patterns. This can be further explored by considering datasets from both technical and non-technical domain and multicolumn tuples containing other sequence to sequence tasks and see which one is more suitable for abstractive text

summarization. According to our study, a conjunction of extractive and abstractive seems to work nicely. It can be interesting to see what other combination works best.

References

[1] Abu Kaisar Mohammad Masum, Sheikh Abujar, Md Ashraful Islam Talukder, AKM Shahariar Azad Rabby, and Syed Akhter Hossain, "Abstractive method of text summarization with sequence to sequence rnns", *10th international conference on computing, communication and networking technologies (ICCCNT)*, pp. 1–5, July 2019.

[2] Rike Adelia, Suyanto Suyanto, and Untari Novia Wisesty, "Indonesian abstractive text summarization using bidirectional gated recurrent unit", *Procedia Computer Science*, pp. 581–588, June 2019.

[3] Shengli Song, Haitao Huang, and Tongxiao Ruan, "Abstractive text summarization using lstm-cnn based deep learning", *Multimedia Tools and Applications*, pp. 857–875, Sept. 2019.

[4] Yong Zhang, Yuheng Wang, Jinzhi Liao, and Weidong Xiao, "A hierarchical attention seq2seq model with copynet for text summarization", *International Conference on Robots & Intelligent System (ICRIS)*, pp. 316–320, 2018.

[5] Chandra Khatri, Gyanit Singh, and Nish Parikh, "Abstractive and extractive text summarization using document context vector and recurrent neural networks", *arXiv preprint*, pp. 7-8, 2018

[6] Anushka Gupta, Diksha Chugh, Rahul Katarya, "Automated news summarization using transformers", *Sustainable Advanced Computing*, pp. 249–259, June 2022.

[7] Fredrik Jonsson, "Evaluation of the transformer model for abstractive text summarization", *Multimedia Tools and Applications*, June 2019.

[8] Urvashi Khandelwal, Kevin Clark, Dan Jurafsky, and Lukasz Kaiser, "Sample efficient text summarization using a single pre-trained transformer", *arXiv preprint*, pp. 905-938, 2019.

[9] Ming-Hsiang Su, Chung-Hsien Wu, and Hao-Tse Cheng, "A two-stage transformer-based approach for variable length abstractive summarization", *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, pp. 2061–2072, 2020.

[10] Elozino Egonmwan and Yllias Chali, "Transformer-based model for single documents neural summarization", *In Proceedings of the 3rd Workshop on Neural Generation and Translation*, pp. 70–79, 2019.

[11] Jie Lin and Saipeng Li, "A generative adversarial constraint encoder-decoder model for the text summarization", *International Symposium on Electrical, Electronics and Information Engineering (ISEEIE)*, pp. 63– 68, 2022.

[12] Lin, Ching-Sheng, "Generative Adversarial Network for Joint Headline and Summary Generation", *arXiv preprint*, pp. 745-751, July 2019.

[13] Liwei Jing, Lina Yang, Xichun Li, and Zuqiang Meng, "Abstractive text summarization using generative adversarial network and relation extraction", *International Conference on Computational Scienceand Computational Intelligence (CSCI)*, pp. 203–206, 2021.

[14] Seyed Vahid Moravvej, Abdolreza Mirzaei, and Mehran Safayani, "Biomedical text summarization using conditional generative adversarial network (cgan)", *arXiv preprint*, pp. 110-118, 2021.